

SIDDAGANGA INSTITUTE OF TECHNOLOGY, TUMAKURU-572103
(An Autonomous Institute under Visvesvaraya Technological University, Belagavi)



Project Report on

“Tree Population Estimation and Classification”

submitted in partial fulfillment of the requirement for the completion of
VII semester of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE & ENGINEERING

Submitted by

Rakesh M S (1SI21CS081)

Vijay M J (1SI21CS120)

Vinay K V (1SI21CS121)

Viresh (1SI21CS123)

under the guidance of

Dr. Srinivasa K

Assistant Professor

Department of CSE

SIT, Tumakuru-03

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

2024-25

SIDDAGANGA INSTITUTE OF TECHNOLOGY, TUMAKURU-572103

(An Autonomous Institute under Visvesvaraya Technological University, Belagavi)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



CERTIFICATE

This is to certify that the project work entitled “[Tree Population Estimation and Classification](#)” is a bonafide work carried out by Rakesh M S (1SI21CS081), Vijay M J (1SI21CS120), Vinay K V (1SI21CS121) and Viresh (1SI21CS123) in partial fulfillment for the completion of VII semester of Bachelor of Engineering in Computer Science & Engineering from Siddaganga Institute of Technology, an autonomous institute under Visvesvaraya Technological University, Belagavi during the academic year 2024-25. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the department library. The Project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the completion of VII semester of Bachelor of Engineering degree.

Dr. Srinivasa K
Assistant Professor
Dept. of CSE
SIT, Tumakuru-03

Dr. N R Sunitha
Head of the Department
Dept. of CSE
SIT, Tumakuru-03

Dr. S V Dinesh
Principal
SIT, Tumakuru-03

External viva:

Names of the Examiners

Signature with date

- 1.
- 2.

ACKNOWLEDGEMENT

We offer our humble pranams at the lotus feet of **His Holiness, Dr. Sree Sree Sivakumara Swamigalu**, Founder President and **His Holiness, Sree Sree Siddalinga Swamigalu**, President, Sree Siddaganga Education Society, Sree Siddaganga Math for bestowing upon their blessings.

We deem it as a privilege to thank late **Dr. M N Channabasappa**, Director, SIT, Tumakuru, **Dr. Shivakumaraiah**, CEO, SIT, Tumakuru, and **Dr. S V Dinesh**, Principal, SIT, Tumakuru for fostering an excellent academic environment in this institution, which made this endeavor fruitful.

We would like to express our sincere gratitude to **Dr. N R Sunitha**, Professor and Head, Department of CS&E, SIT, Tumakuru for her encouragement and valuable suggestions.

We thank our guide **Dr. Srinivasa K**, Assistant Professor, Department of Computer Science & Engineering, SIT, Tumakuru for the valuable guidance, advice and encouragement.

Rakesh M S (1SI21CS081)

Vijay M J (1SI21CS120)

Vinay K V (1SI21CS121)

Viresh (1SI21CS123)

Course Outcomes

After successful completion of major project, graduates will be able to:

CO1: To identify a problem through literature survey and knowledge of contemporary engineering technology.

CO2: To consolidate the literature search to identify issues/gaps and formulate the engineering problem

CO3: To prepare project schedule for the identified design methodology and engage in budget analysis, and share responsibility for every member in the team

CO4: To provide sustainable engineering solution considering health, safety, legal, cultural issues and also demonstrate concern for environment

CO5: To identify and apply the mathematical concepts, science concepts, engineering and management concepts necessary to implement the identified engineering problem

CO6: To select the engineering tools/components required to implement the proposed solution for the identified engineering problem

CO7: To analyze, design, and implement optimal design solution, interpret results of experiments and draw valid conclusion

CO8: To demonstrate effective written communication through the project report, the one-page poster presentation, and preparation of the video about the project and the four page IEEE/Springer/ paper format of the work

CO9: To engage in effective oral communication through power point presentation and demonstration of the project work

CO10: To demonstrate compliance to the prescribed standards/ safety norms and abide by the norms of professional ethics

CO11: To perform in the team, contribute to the team and mentor/lead the team

CO-PO Mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO-1											3			
CO-2		3												
CO-3										3			3	
CO-4						3							3	
CO-5	3	3											3	
CO-6					3						3		3	
CO-7			3	3									3	
CO-8									3					
CO-9									3					
CO-10							3							
CO-11								3						
Average	3	3	3	3	3	3	3	3	3	3	3		3	

Attainment level: 1: Slight (low) 2: Moderate (medium) 3: Substantial (high)

PO1: Engineering knowledge.

PO2: Problem analysis.

PO3: Design of solutions.

PO4: Conduct investigations of complex problems.

PO5: Engineering tool usage.

PO6: Engineer and the world.

PO7: Ethics.

PO8: Individual and collaborative work.

PO9: Communication.

PO10: Project management and finance.

PO11: Life-long learning.

PSO1: Computer-based systems development.

PSO2: Software development.

PSO3: Computer communications and Internet applications.

Abstract

In this work, a system is designed to streamline the process of plant and tree identification through automated image analysis. This system has a simple but effective platform to upload images, which are instantly analyzed and classified. The mechanism for uploading an image is user-friendly and consists of processing an uploaded photo by detecting and classifying the different kinds of plants and trees in the scene.

The system is designed to be simple and accessible, offering an easy-to-use platform for environmental researchers, urban planners, and nature enthusiasts. It uses advanced technologies like YOLOv9 for object detection and Convolutional Neural Networks (CNN) for species classification, ensuring accurate results even in challenging conditions like varying light or overlapping trees.

The system provides comprehensive analysis results, including tree detection, classification and total count of trees or plants identified. Users can preview their uploaded images and receive organized analytical results for further use through the streamlined interface. This makes it particularly valuable for enthusiasts, managers, and environmental researchers who need quick and accurate plant identification and quantification.

Contents

Abstract	i
List of Figures	ii
List of Tables	iii
1 Introduction	1
1.1 Motivation	2
1.2 Objectives	3
1.3 Organisation of the report	3
2 Literature Survey	5
2.0.1 Summary	12
3 System Overview	17
4 System Architecture and High Level Design	20
4.1 System Architecture	20
4.2 High Level Design	25
4.3 About Functional Requirements	29
4.4 About Non Functional Requirements	32
4.5 Software and Hardware Requirements	35
4.5.1 Software Requirements	35
4.5.2 Hardware Requirements	36
5 Software Architecture and Low Level Design	38
5.1 About the Algorithm	38
5.2 UML Diagrams	43
5.2.1 Use case diagram	43
5.2.2 Sequence Diagram	45
5.2.3 Activity Diagram	47

6	Results	49
6.1	Test Setup Environment	49
6.2	Test Procedures and Test Cases	49
6.3	Snapshots	50
6.4	Analysis	51
7	Conclusion	53
7.1	Scope for future work	53
	Bibliography	54
	Appendices	57
A	Project Management and Budget Estimation Details	58
B	Configuration Details	60
C	Data Set Details	61
D	Code Snippets	62

List of Figures

3.1	System Overview of Tree Counting and Classification	17
4.1	Plant Counting and Classification System Architecture with CNN	20
4.2	High Level Design of Tree Population Eastimation and Counting	25
5.1	Low Level Design	39
5.2	Usecase Diagram	43
5.3	Sequence Diagram	45
5.4	Activity Diagram.	47
6.1	Model Plot	50
6.2	Web Interface for Tree Counting and Classification (1)	51
A.1	Project Plan	58

List of Tables

4.1	Summary of Application Layers and Components	24
-----	--	----

Chapter 1

Introduction

Tree population estimation and classification have become essential in environmental studies, aiding researchers, conservationists, and urban planners in effective decision-making. Its significance lies in providing accurate insights into tree distribution, species diversity, and ecological health. Advanced AI-driven solutions now enable automated detection, classification, and analysis of trees, streamlining the traditionally labor-intensive process. The tree population estimation and classification system utilizes artificial intelligence and deep learning technologies to process images from various environments, including gardens, parks, forests, and urban landscapes. Traditional methods of tree identification and counting are often tedious, error-prone, and unsuitable for large-scale analysis. By incorporating object detection algorithms such as YOLOv8 for real-time tree detection and CNNs for species classification, this system enhances efficiency and accuracy. Designed with practical usability in mind, the system features a user-friendly interface, making it accessible to professionals and enthusiasts alike. The system facilitates ecological studies, biodiversity monitoring, and conservation planning while supporting applications in urban tree management, forestry, and agriculture.

The system can identify tree species by analyzing key visual features such as leaf structure, fruit characteristics, and canopy patterns even in challenging conditions. Additionally, the platform produces comprehensive reports that can be downloaded for further analysis, thereby enhancing its utility in academic research and professional studies. The project is not only a technical innovation but also a practical tool for environmental management. The solution plays a pivotal role in supporting urban planning by helping assess green cover, monitoring deforestation, and identifying endangered species. Its applications in biodiversity research provide researchers with valuable data to understand ecosystems better and implement conservation measures effectively. With rapid urbanization and climate change, tools such as the Tree Population Estimation and Classification project are invaluable. These advancements help facilitate data-driven decision-making for sound environmental practices.

Through the process of automated analysis of trees, the initiative serves as evidence that shows how AI can be used in dealing with global ecological issues and aid more sustainable development strategies. With cutting-edge technologies and a scalable, efficient system, the Tree Population Estimation and Classification project contributes highly to ecological research and environmental conservation. The development represents one of the most notable ways AI can be used to tackle real-world issues and help to bridge the divide between technology and nature. This advancement will serve as a benchmark for future innovations in environmental monitoring and will also remind the world to adopt AI-based solutions for a sustainable future.

1.1 Motivation

The motivation is fueled by the limitations of existing techniques and the increasing need to base the approach on data-driven strategies towards conserving the environment. The assessment of green cover, deforestation monitoring, and studies on the distribution of tree species require intensive and reliable tools at the disposal of city planners, ecologists, and researchers. The recent improvement in artificial intelligence and deep learning has created the potential for automation, being more accurate, efficient, and accessible than the previous methods. With real-time object detection through state-of-the-art technology YOLOv8 and species classification with CNN, it addresses challenges like occlusion, varying lighting conditions, and cluttered backgrounds in performance, thereby guaranteeing robustness in the system for environments that range from dense forests to busy streets.

Recent improvements in artificial intelligence (AI) and deep learning have made it possible to automate these tasks more accurately, efficiently, and easily. With technologies like YOLOv8 for real-time object detection and CNN for classifying tree species, we can overcome challenges like trees being hidden by other objects, changing light, or messy backgrounds. This ensures the system works well in different environments, from dense forests to busy city streets. Furthermore, using AI helps create faster, more precise tools for managing and conserving our natural resources. These tools can be used for various purposes like city planning, protecting endangered species, or monitoring changes in forests. By using these advanced technologies, we can better understand our environment and make informed decisions to protect it.

1.2 Objectives

1. To Develop an Efficient and Accurate Tree Counting System: A robust tree counting application is to be developed, with tree detection and counting conducted over ground-level images within a YOLOv8 framework. This system should overcome problems from occlusion through robust data augmentation and feature extraction methods. The training should use data from different areas, seasons, and plant types to make the model work better in all situations. A method for detecting trees of different sizes and growth stages is to be added.

2. To Implement Leaf-Based Tree Species Classification Using CNN: A CNN-based system is to be developed to accurately identify tree species using visual features like leaves, fruits, bark patterns, and canopy shape. The system should achieve high accuracy across different environments, support the easy addition of new species, and integrate with tree detection for a complete tree identification and classification solution.

1.3 Organisation of the report

The report is structured to give a comprehensive understanding of the Tree Population Estimation and Classification project, including its purpose, methodology, and outcomes. It begins with an introduction that underscores the importance of the project in the resolution of ecological challenges and environmental conservation and sustainable development. The Motivation section is an elaboration of the necessity for automated tools to estimate and classify tree populations efficiently, bypassing the inefficiencies of conventional approaches. Subsequently, the Literature Survey reviews the relevant research and technology, providing a solid basis for the approach that has been taken and pointing out gaps that this project addresses. The Objectives section defines key objectives, emphasizing automation, species classification, scalability, and real-world applicability.

The report will construe the theoretical framework, ranging from YOLOv8 usage for detection to CNNs for classification, thus boasting data augmentation techniques to uplift the model's reliability. Implementation deals with tools, technologies, and techniques used for reproducibility and adaption. Results of performance measures, accuracy, and real-world applications of the system are illustrated in the Results and Analysis section

of the report. Applications of biodiversity monitoring and urban forestry are described in terms of practicality in the Applications section. The report is concluded with a Conclusion which elaborates on the contributions and the impact of the project and frames future possibilities for improvements. All references utilized in writing this report are being stated at the end. The present systematic and coherent arrangement contributes to utmost clarity and comprehensiveness, thus making the report accessible to a vast audience.

In summary, this chapter has provided an overview of the Tree Population Estimation and Classification System, highlighting its significance and practical applications. The motivation behind the project, rooted in the need for accurate and scalable tree analysis solutions, has been discussed, along with the core objectives that drive this research. The subsequent chapters will delve deeper into the literature, methodologies, system architecture, and implementation, providing a comprehensive exploration of the project's development.

Chapter 2

Literature Survey

To develop an efficient tree estimation and classification system, it is crucial to examine existing research and technological advancements in the field. This chapter reviews relevant literature on object detection, image classification, and tree population analysis using artificial intelligence. The discussion encompasses various methodologies and algorithms previously implemented, their advantages and limitations, and how they contribute to the foundation of this project.

Fast region-based CNNs tend to rely on GPUs, while region-proposal methods are often CPU-based, making objective run-time comparisons challenging. An engineering solution may have been to reimplement these proposals for GPUs, but this often leads to overlooking the synergy between proposal generation and detection. As suggested in [1], the authors introduce Region Proposal Networks (RPNs), which generate proposals using deep convolutional networks while sharing convolutional layers with state-of-the-art object detection networks. By doing this, the marginal cost of computing these proposals is reduced to approximately 10 ms per image. The RPNs produce region proposals on the same convolutional feature maps used by detectors such as Fast R-CNN, with some extra convolutional layers to simultaneously regress the region boundaries and the objectness score.

Automating tree counting through image processing and machine learning leverages data obtained from unmanned aerial vehicles (UAVs), which are widely used across various fields with proper authorization. As highlighted in [2], the DJI Advanced 4 Pro UAV was employed to capture images from a specific altitude and analyze them using ground station software. Key features extracted using Scaled Invariant Feature Transformation (SIFT) were processed with a pre-trained Extreme Learning Machine (ELM) classifier to detect palm trees. This approach enhances accuracy and efficiency in tree detection, demonstrating the potential of UAV-based systems for large-scale environmental moni-

toring and forestry applications.

Satellite-based tree counting utilizes YOLO v5, UNET, and SSD to enhance forest management by addressing limitations in manual counting methods. Traditional approaches often suffer from inaccuracies, while satellite-based methods encounter challenges such as cloud cover and low image resolution. As discussed in [3], dataset preparation involved bounding box annotation and augmentation techniques like rotation, flipping, and patch division, improving model performance by enhancing dataset diversity and reducing image size. A comprehensive analysis of related research, methodology, and experimental results highlights the effectiveness of object detection and segmentation models for tree counting. Challenges associated with satellite-based analysis, along with potential improvements, are also addressed. Findings emphasize the advantages of deep learning techniques in automating tree detection, showcasing their ability to overcome existing limitations. Future research aims to refine accuracy and mitigate data constraints, contributing to more reliable and scalable solutions for forest monitoring.

CNNs with deep learning models have been explored for classifying and counting trees using digital image processing techniques. A CNN model was trained on two distinct datasets, including images both with and without trees, incorporating positive and negative examples. As outlined in [4], MATLAB and the Parallel Computing Toolbox were utilized to enhance processing speed during training. A comprehensive overview of digital image processing and deep learning methodologies for tree detection and counting is provided, covering three different counting methods and evaluating their effectiveness. Background and existing techniques relevant to these approaches are examined, detailing the proposed method that integrates CNNs with digital image processing. Additionally, a comparative analysis of results obtained from different counting methodologies is presented, demonstrating the strengths and limitations of each approach.

Deep learning-based tree detection and counting using Convolutional Neural Networks (CNNs) have been explored in ground-level imagery, particularly for dense urban environments where trees may be obscured by buildings, vehicles, and other objects. Traditional detection methods often struggle in such scenarios, while CNNs enable the identification

and counting of individual trees even in complex and cluttered settings. As demonstrated in the research, deep learning models significantly enhance tree detection and counting performance in environments where conventional computer vision techniques typically fall short. According to [5], the proposed approach is particularly effective in handling overlapping or occluded trees, a common challenge in cityscapes. This methodology offers a valuable tool for real-time monitoring of urban green spaces, contributing to applications in urban planning, environmental monitoring, and biodiversity conservation.

Class-CNNs (Convolutional Neural Networks) have been explored for identifying and counting tree species from ground-level images by integrating classification and counting tasks within a single module. Researchers have utilized CNNs to efficiently recognize tree species based on visual features such as leaf structure and bark texture while also estimating the number of trees present. As detailed in [6], this integrated approach enhances tree-monitoring efficiency, particularly in cases where frontal views are available but are complicated by factors such as variable lighting, occlusions, and busy backgrounds. The proposed model is designed for large-scale real-world applications, proving beneficial for urban forestry and ecological studies that estimate tree population density and species diversity. This research highlights how deep learning improves both accuracy and scalability, providing a practical solution for tree detection and classification in diverse environments.

Tree species classification is crucial for forest management, biodiversity conservation, and ecological studies. Remote sensing technologies, such as passive optical, LiDAR, and hyperspectral imaging, provide enhanced spatial and spectral resolutions for effective classification. As noted in [7], recent advancements in deep learning, particularly convolutional neural networks (CNNs), have automated feature extraction, significantly improving classification accuracy. While unimodal data methods, like hyperspectral imaging, excel at capturing spectral details, multimodal approaches that integrate diverse datasets (e.g., LiDAR and RGB) offer more robust classification. However, challenges remain in dealing with overlapping canopies, integrating spatial-temporal data, and accounting for phenological variations. Future research aims to refine data fusion techniques and utilize seasonal characteristics to improve precision.

Discriminating tree species using remote sensing is essential for managing both urban and rural forests due to its ecological and economic significance. Advances in high-resolution satellite imagery, such as WorldView-3 (WV-3), along with machine learning techniques like Random Forest (RF) classifiers, have enhanced classification accuracy. As discussed in [8], multi-seasonal and multi-temporal data, which capture key phenological phases, are critical for improving spectral discrimination of species. However, challenges remain with species that have similar spectral responses and in refining phenology-based classification. Future research must focus on optimizing phenological insights and robust data fusion techniques to enable accurate and scalable classification.

Mapping tree species diversity plays a vital role in maintaining ecosystem services and supporting biodiversity conservation. While remote sensing methods, especially hyper-spectral and LiDAR data, have proven effective in predicting species richness, they are often limited by resolution constraints. According to [9], recent advancements in very high-resolution (VHR) RGB imagery offer a more detailed examination of canopy and forest structures. Research has highlighted the effectiveness of object-based and patch-based techniques using VHR imagery for detecting individual trees and identifying species. Deep learning methods, such as CNNs (e.g., VGG16, ResNet50), provide superior accuracy in feature extraction compared to conventional approaches. Nevertheless, challenges remain in managing spectral variation and texture details in complex environments. Combining LiDAR-derived canopy height models with deep learning techniques has shown promise in improving mapping and classification accuracy, as evidenced in applications to tropical wetlands and urban forests.

Tropical montane grasslands (TMG) are ecologically significant but are increasingly threatened by exotic tree invasions, especially species like Acacia and Pines, which disrupt ecosystems worldwide. As noted in [10], remote sensing has become a crucial tool for monitoring and managing these invasions over vast and remote areas. Traditional pixel-based classification struggles with high-resolution images, often resulting in issues such as salt-and-pepper noise. Instead, segmentation of homogeneous regions proves more effective in distinguishing complex landscape features. Remote sensing plays a key role in restoration efforts by identifying potential sites and assessing invasion drivers, such as

proximity to roads, soil properties, and fire history. By providing detailed insights into vegetation dynamics, this method also aids in understanding the patterns of tree invasions over time. This approach improves site identification and informs strategies to combat invasions, thereby supporting grassland restoration initiatives.

Species abundance models (SAMs) are designed to estimate the relative population of species by incorporating ecological variables, offering more comprehensive insights compared to traditional species distribution models (SDMs). While SDMs are typically reliable in predicting species presence, SAMs struggle with abundance due to the intricate nature of environmental gradients and geographical factors. As mentioned in [11], studies using models like Random Forests and other statistical approaches have produced inconsistent results, with performance varying across different species and regions. Elements such as weather conditions, soil attributes, and species-specific characteristics, such as dominance and spatial patterns, impact the precision of the models. Integrating factors like northern latitude boundaries and a diverse set of predictors has increased the reliability of SAMs, particularly for dominant species. However, interactions between species and ecological factors often obscure abundance trends, emphasizing the need to refine SAM methods for broader application in large-scale biodiversity and climate studies.

Advances in remote sensing and machine learning have led to the development of broad-scale species classification models, expanding their application beyond site-specific contexts. As outlined in [12], using hyperspectral data from NEON, efforts are focused on classifying tree species at the individual crown level across U.S. ecosystems. General models outperform site-specific classifiers, with a micro-F1 score of 0.77 compared to 0.70, and aggregating data to the genus level further enhances accuracy to 0.83. However, challenges persist, such as misclassification of closely related species, phenological variations, and spectral inconsistencies. By integrating data from multiple sites and employing ensemble methods, the models' predictions and uncertainty estimates are strengthened. These models hold significant potential for scaling ecological studies and biodiversity monitoring across extensive landscapes.

Standing dead trees (snags) are crucial indicators of forest biodiversity and ecosystem health, providing essential habitats for snag-dependent species like the Black-backed Woodpecker. As discussed in [13], Airborne Laser Scanning (ALS) has become a powerful tool for detecting and mapping snags and their spatial distribution, particularly in burned forests. Recent studies reveal that ALS-derived metrics, such as height, intensity, and crown geometry, can effectively classify snags and estimate basal area with high precision. The integration of Gaussian process models further refines these predictions by incorporating ALS data. Applications in post-fire areas, such as California's Rim Fire, highlight the potential of ALS for mapping habitat suitability and informing forest management decisions. Despite advancements in ALS technology, challenges remain in fine-tuning snag classification across diverse forest conditions and combining ALS data with other ecological information.

Climate-induced bark beetle outbreaks are a growing threat to temperate forests across Europe, calling for innovative approaches to forest management. According to [14], Unmanned Aerial Systems (UAS) equipped with multispectral and RGB sensors have proven highly effective for classifying tree species and assessing forest health with high spatial resolution. Methods like structure-from-motion (SfM) and specialized spectral indices, such as PSRI and NDRE, enhance the detection of stress in tree canopies. Research incorporating texture analysis and support vector machines (SVM) has shown improved accuracy in identifying tree species and pinpointing unhealthy or dead trees. Despite advancements, adapting these techniques to dense, complex forest structures remains challenging. By combining spectral, textural, and statistical methods, these gaps can be addressed, providing valuable insights for species classification, bark beetle infestation detection, and sustainable forest management strategies.

In North America, forest tree species are under threat from invasive pests and diseases, leading to substantial ecological and economic challenges. In [15], the Project CAPTURE framework addresses this by assessing the vulnerability of 419 tree species. By applying K-means clustering, the framework categorizes species into 11 groups based on three criteria: exposure to invasive threats, susceptibility to damage, and potential for resilience. This method, which combines empirical data with expert judgment, highlights species needing

immediate protection, such as the American chestnut and ash, which are severely affected by invasive pests like chestnut blight and emerald ash borer. Additionally, the framework identifies species that could benefit from breeding for resistance and those requiring ongoing monitoring for future threats. By evaluating species characteristics alongside the intensity of risks, Project CAPTURE aids in optimizing conservation efforts, ensuring the protection of genetic diversity vital for maintaining ecosystem function and resilience.

Remote sensing-based tree species classification has significantly advanced over the past four decades, with many studies focusing on the use of various sensor types. However, as noted in [16], a comprehensive review that addresses the effectiveness of these sensors, classification methods, and the challenges of large-area classifications is still missing. Most research has concentrated on local-scale approaches, with limited examples of applications on larger geographic scales. While data-driven techniques have been widely used to optimize classification accuracy, few studies have focused on hypothesis-driven research. Additionally, there is a lack of studies examining the traits influencing remote sensing signals that affect species identification. Future research should aim to understand the underlying causes of classification success or failure across different conditions and reduce the reliance on data-driven studies that lack thorough experimental designs.

Individual Tree Detection-and-Counting (ITDC) is crucial for urban environments, where numerous methods have been suggested, though many are still inadequate due to their reliance on field investigations. As discussed in [17], this paper introduces a novel approach combining high-resolution imagery and Canopy-Height-Model (CHM) data to enhance ITDC. The method is applied across six urban settings, including farmland, park, and industrial areas, using a deep learning network to identify tree canopy regions. The CHM data is then utilized to detect treetops and individual tree canopies using a local maximum algorithm and region growing, improving accuracy and efficiency in tree detection. Additionally, this combined approach offers a more scalable solution compared to traditional field-based methods, significantly reducing the need for labor-intensive surveys. Tested in Shanghai, this method achieved an average accuracy of 0.953, with particularly high precision in woodland areas.

Through this literature review, we have identified gaps in existing tree population estimation techniques and explored advanced deep learning methodologies that address these challenges. The insights gained from this review play a crucial role in shaping the system's design, ensuring its effectiveness in real-world applications.

2.0.1 Summary

Title	Conference/ Journal	Methodology	Limitation
Tree Detection and Counting from an Image	Department of Forest Engineering at Oregon State University	Used CNNs to detect and count trees in dense urban settings with occlusions, outperforming traditional methods.	Limitations include environmental sensitivity, occlusions, scale variability, and real-time processing challenges.
Automating Tree Counting Using UAVs and Machine Learning	Siirt University Scientific Research Projects Coordination (SIUBAP), UFEB-21.	The study utilized UAV images, processed with SIFT and classified using the ELM classifier for automated tree counting.	The study was limited to a specific UAV model and did not explore more advanced machine learning techniques or real-world application challenges.
Satellite-Based Tree Counting Using Deep Learning Models	Journal of Applied Remote Sensing, Feb. 2023	Discussed training of object detection and image segmentation models (YOLO v5, UNET, SSD) for satellite-based tree counting.	The study faced challenges like cloud cover, low image resolution, and limited availability of images, which caused low accuracy for the models.

Digital Image Processing and Deep Learning for Tree Classification	International Journal of Computer Vision	Explored how CNNs work with deep learning models to classify and count trees using digital image processing techniques.	The study was limited to two distinct datasets and did not explore more advanced machine learning models or real-world application challenges.
Tree Detection and Counting from Ground-Level Imagery Using Deep Learning	Journal of Urban Forestry	Introduced a deep learning-based approach for tree detection and counting using CNNs in ground-level imagery, particularly designed for dense urban settings.	The study did not explore the use of more advanced machine learning models or real-world application challenges.
Using Deep Learning for Tree Species Identification and Counting from Ground-Level Images	Journal of Environmental Monitoring	Combined classification and counting tasks within the same module using Class-CNNs for identification of tree species and counting from ground-level images.	The study was limited to specific datasets and did not explore integration with emerging technologies or real-world application challenges.

Tree Species Classification Based on Remote Sensing Data and Deep Learning-Based Methods	Forests, Multidisciplinary Digital Publishing Institute (MDPI) 2024	CNNs for tree species classification using remote sensing data, leveraging spatial and spectral resolution for improved precision.	Tree Species Classification & Handling canopy overlap, spatial-temporal data integration, and phenological variations
Discriminating Tree Species at Different Taxonomic Levels Using Multi-Temporal WorldView-3 Imagery in Washington DC, USA	Remote Sens. Environ., 2020	Multi-seasonal and multi-temporal WorldView-3 imagery combined with Random Forest classifiers for tree species discrimination.	Species with similar spectral responses, refining phenology-based classification
Tree Species Mapping with LiDAR	Forests, 2019	Hyperspectral and LiDAR data with deep learning (VGG16, ResNet50) for mapping tree species diversity.	Spectral variation and texture details in complex scenes
Opportunities and Challenges in Using Remote Sensing for Invasive Tree Species Management, and in the Identification of Restoration Sites in Tropical Montane Grasslands	J.Environ. Manag., 2021	Object-oriented classification with high-resolution remote sensing to monitor invasive species and support restoration.	Salt-and-pepper noise from pixel-based classification, large-scale implementation issues

Challenges in Modelling the Abundance of 105 Tree Species in Eastern North America Using Climate, Edaphic, and Topographic Variables	For. Ecol. Manag., 2013	SAMs combined with environmental variables and Random Forest models to predict tree species abundance.	Environmental gradient complexities and ecological process obfuscation
Continental-Scale Hyperspectral Tree Species Classification in the United States National Ecological Observatory Network	Remote Sens. Environ., 2022	Hyperspectral data from NEON, ensemble methods for classifying tree species across diverse U.S. ecosystems.	Misclassification among congeneric species, spectral discrepancies across regions
Burned Forest Characterization at Single-Tree Level with Airborne Laser Scanning for Assessing Wildlife Habitat	Remote Sens. Environ., 2016	Airborne Laser Scanning (ALS) combined with Gaussian process models for detecting snags in burned forests.	Refining snag classification under variable forest conditions
Tree Species Classification and Health Status Assessment for a Mixed Broadleaf-Conifer Forest with UAS Multispectral Imaging	Remote Sens., 2020	UAS multispectral and RGB imagery with SVM for tree species classification and bark beetle health assessment.	Adapting methods to dense and structurally complex forests

Prioritizing the Conservation Needs of United States Tree Species: Evaluating Vulnerability to Forest Insect and Disease Threats	Glob. Ecol. Conserv., 2019	K-means clustering for categorizing tree species based on vulnerability to invasive insect and disease threats.	Incomplete data on pest impacts and species adaptive traits
Review of Studies on Tree Species Classification from Remotely Sensed Data	Remote Sens. Environ., 2016	Review of sensor types and classification methods for large-area tree species identification and challenges.	Limited studies on large-area classification, lack of hypothesis-driven research
Individual Tree Detection-and-Counting Based on High-Resolution Imagery and the Canopy Height Model Data	Geo-Spatial Information Science, 2024	Deep learning combined with CHM data for individual tree detection and counting in urban environments.	Challenges in non-urban environments, requiring refinement for complex scenes

This table summarizes the key findings from various studies related to tree population estimation and classification. It outlines the methodologies used, the limitations encountered, and the significant contributions of each study. This table provides a comprehensive overview of the current state of research in this field, highlighting the advancements and challenges in tree detection and classification using different techniques and technologies.

Chapter 3

System Overview

This chapter presents a comprehensive examination of the system architecture and processing pipeline for tree population estimation and classification. The chapter details the integration of various components, from data input through to user interface presentation, highlighting the role of advanced machine learning techniques in achieving accurate species identification and counting.

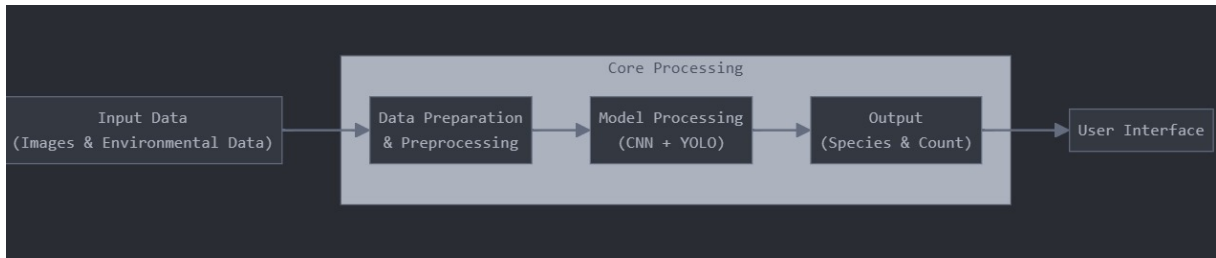


Figure 3.1: System Overview of Tree Counting and Classification

Figure 3.1 describes the process flow for data input, preprocessing, model processing, and output generation. The system takes in images and environmental data, processes the input through data preparation and model processing using CNN and YOLO, and produces species identification and count as output, which is then displayed via a user interface.

Input Data (Images & Environmental Data)

The process begins with the **Input Data** stage, which combines visual and environmental information to provide a comprehensive dataset. Images are captured by cameras or sensors and may include objects of interest such as wildlife, plants, or other entities. These images are complemented by environmental data, such as GPS coordinates, weather conditions (e.g., temperature, humidity), and timestamps, which offer valuable context. For example, the time of day or specific environmental conditions can significantly influence the interpretation of images, especially in wildlife or ecological studies. This integration of visual and non-visual data ensures that the subsequent stages are well-informed and robust.

Data Preparation & Preprocessing

The next stage is **Data Preparation & Preprocessing**, where raw input data undergoes various transformations to make it suitable for analysis. This stage begins with **data cleaning**, which removes noise, irrelevant data, and inconsistencies such as blurry images or corrupted files. For images, preprocessing includes **normalization**, where pixel values are scaled to a uniform range (e.g., 0 to 1), and **data augmentation**, where techniques such as rotation, flipping, and cropping are applied to artificially expand the dataset. These augmentations increase the diversity of the training data, improving the model's robustness and accuracy. Additionally, environmental data is encoded into machine-readable formats, such as numerical or categorical representations, ensuring seamless integration with machine learning models. If necessary, **segmentation** is performed to highlight regions of interest in the images, reducing the computational load during model processing.

Core Processing

At the core of the pipeline is the **Core Processing** stage, where advanced machine learning models analyze the prepared data. Two key components in this stage are **Convolutional Neural Networks (CNNs)** and **You Only Look Once (YOLO)**. CNNs are used for feature extraction, identifying patterns, shapes, and other significant characteristics within the images. These extracted features are then passed to YOLO, a real-time object detection algorithm that processes entire images in a single pass. YOLO detects multiple objects simultaneously and outputs their bounding boxes along with class labels. By combining the feature extraction capabilities of CNNs with the real-time detection efficiency of YOLO, the pipeline achieves both high accuracy and speed, making it suitable for tasks like species identification and tracking in dynamic environments.

Output (Species & Count)

The results of the core processing are presented as the **Output**, which provides a structured summary of detected objects. The system identifies the species in the images and calculates their respective counts. For instance, if an image contains lions and zebras, the output might specify "2 Lions, 3 Zebras." Each detected object is labeled and localized using bounding boxes, ensuring clarity and precision. This output serves as the founda-

tion for further analysis, enabling users to derive actionable insights, such as monitoring biodiversity or evaluating population trends in ecological studies.

User Interface

The final stage is the **User Interface**, where the processed results are displayed to end-users in an interactive and accessible manner. The interface typically includes annotated images with labeled bounding boxes to highlight detected objects visually. Additionally, it may present statistical summaries, such as species counts and trends, in tables or graphs. Supplementary information, such as timestamps or environmental conditions, is also displayed to provide context. The interface is designed to be user-friendly, ensuring that both technical and non-technical users can easily interpret and interact with the data. Features like search functionality, filtering, or exporting results further enhance usability, making the system practical for real-world applications.

Summary

In conclusion, this pipeline integrates multiple stages—from raw data input to user-friendly output presentation—to deliver accurate and actionable results. The seamless combination of visual and environmental data, advanced preprocessing techniques, and state-of-the-art machine learning models ensures robust performance. By presenting the output through an intuitive interface, the pipeline facilitates meaningful insights and supports decision-making across various domains, such as wildlife conservation, ecological monitoring, and resource management.

The chapter presented a detailed overview of the system architecture, from input processing through to user interface design, with particular emphasis on the integration of CNN and YOLO technologies for accurate tree identification and counting. This foundational understanding sets the stage for detailed implementation considerations and performance analysis, which will be explored in subsequent chapters.

Chapter 4

System Architecture and High Level Design

4.1 System Architecture

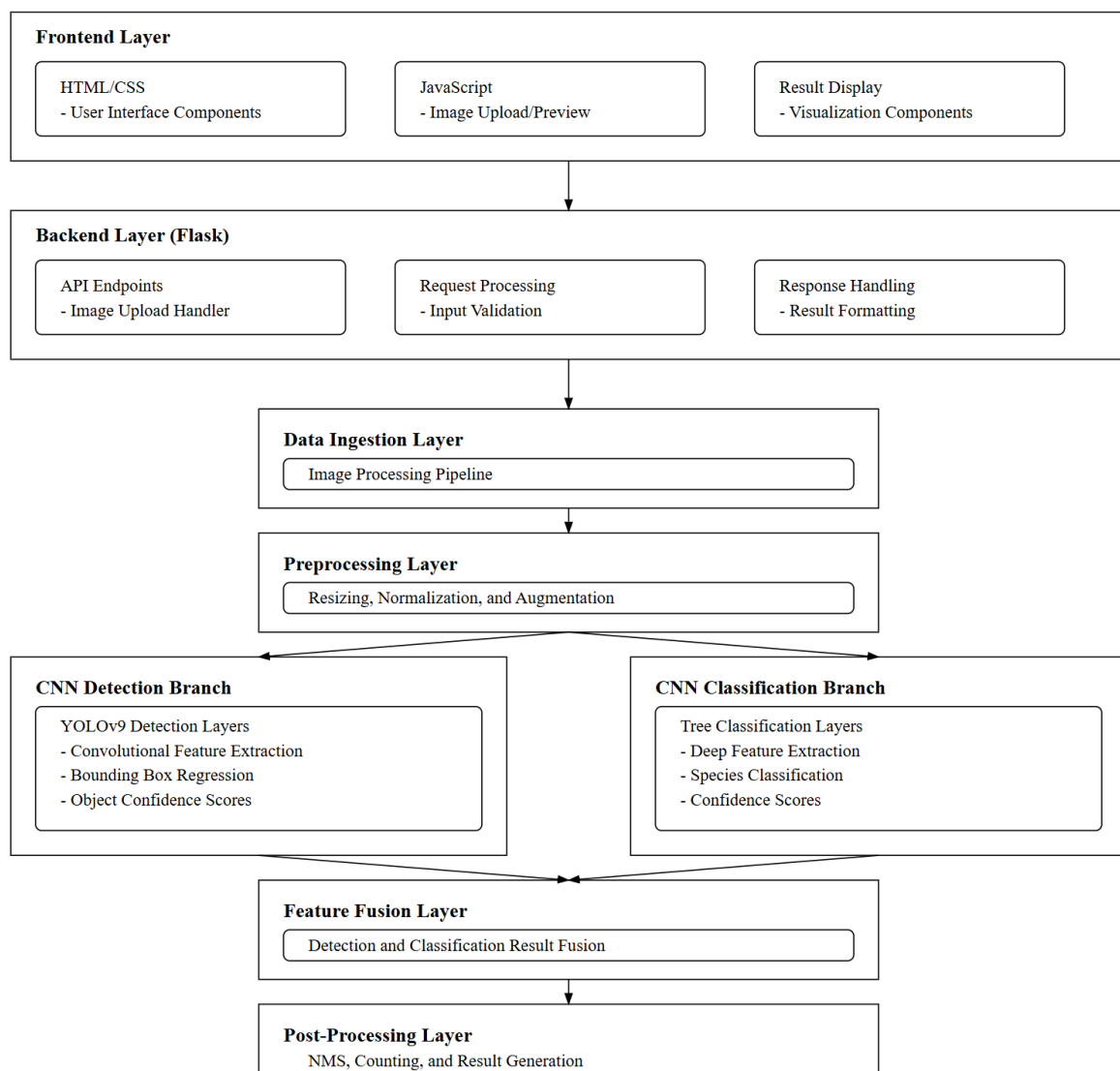


Figure 4.1: Plant Counting and Classification System Architecture with CNN

A well-structured system architecture is essential for the seamless functioning of a tree detection and classification model. This chapter presents the high-level design of the system, outlining its major components and how they interact. By leveraging advanced artificial intelligence techniques, the system is designed to efficiently detect, classify, and analyze trees from images, ensuring accuracy and scalability.

Figure 4.1 illustrates the system architecture, outlining the organization and interaction of its core components, defining how user requests are processed, tree detection and classification are performed, and data is managed. This architectural overview provides a clear understanding of the system's structure and the responsibilities of each component, which is described in detail below:

Frontend Layer: The frontend layer provides the user interface and ensures a seamless and engaging experience. Built with HTML and CSS, it offers a responsive design that adapts to various devices, including desktops, tablets, and mobiles. JavaScript enables dynamic features such as live image previews and real-time validations, using visualization libraries like D3 or Chart.js. Advanced frameworks like React can be employed for better state management and modularity. Features like drag-and-drop upload, tooltips, and live feedback improve accessibility. Real-time updates via WebSocket or polling enhance interactivity, allowing users to receive instant responses without needing to refresh the page.

Backend Layer: The backend layer is basically the core communication hub built with Flask to link the frontend with the sophisticated backend of the system. This layer is responsible for maintaining RESTful API endpoints for image uploads, preprocessing spot signification, and dispatching results. To be sure that the user inputs adhere to the system requirements, while managing gracefully for errors with clear feedback messages. Flask makes it all possible to integrate well with machine-learning models, whereas an asynchronous task manager around Celery allows the system to processor requests with higher volumes. Other plans put in place for improved security require token-based authentication and encrypted-file storage for the protection of user data through the lifecycle of the tool. Built-in logging and monitoring mechanisms in the backend ensure that the system performance can be logically tracked, while any errors can easily be solved and a detailed analysis provides insight for future refinements.

Data Ingestion Layer: The data ingestion layer is the first place where processing takes

place, acting as the gateway to ensure that the uploaded images are properly readied for analysis. This layer verifies the file format, resolution, and size of the uploaded images and saves them temporarily on the server, logging all relevant metadata for reference. By organizing and validating the inputs at this point, the system keeps a strong flowing work process to accommodate high user traffic. Also, this layer implements queuing mechanisms to prioritize and handle large datasets. Automated data cleaning processes, such as the removal of unsupported or corrupt files, minimize disruptions in the pipeline. The system also tracks and analyzes input statistics, providing insights into usage patterns that help optimize resource allocation and system performance.

Preprocessing Layer: This preprocessing layer is meant to polish and prepare any and all images for analysis. They first resize everything generally to the dimensions recommended by the YOLOv9 model. Aspect ratios should also be preserved to prevent distortion. The technique of normalizing is intended to scale down each pixel value to ensure data fits in a standard range. This is to keep it in small scope and have all the data on an equal scale so that this helps to improve model performance when averaged over them. More input data are generated from the data-augmentation techniques such as rotation, flipping, cropping, and brightness adjustment in order to make the models more versatile concerning the variations. This layer works in high-throughput, batch-processing hundreds of thousands of images from some datasets and is tuned to ensure reliable performance under even high-demand situations.

CNN Detection Branch: The detection branch employs YOLOv9 for the detection of objects in uploaded images. The branch extracts spatial features using convolutional layers and provides bounding box regression outputs along with a score indicating how reliable the detection is. The grid-based system of YOLOv9 enables real-time detection while preserving accuracy, making it applicable to cases of too many objects and high resolutions. Such advanced methods that increase its capability to handle overlapping or partially occluded detections include anchor-free detection and improved loss functions. Other robust mechanisms overcome the challenges posed by occlusions and dense arrangements by ensuring that identification operates even in difficult environments. Its modular design allows hyperparameters to be tuned easily, tailoring the system for specific datasets or applications.

CNN Classification Branch: The classification branch is responsible for identifying

the categories or species of detected objects, which is essential for distinguishing different tree species or other objects in the image. It uses deep convolutional layers to extract high-level features like texture, shape, and patterns for accurate classification. These features are then processed through fully connected layers to predict the category label for each detected object, with each result accompanied by a confidence score that indicates the model's certainty. Transfer learning, utilizing pretrained models like ResNet or Inception, enhances accuracy by leveraging proven feature extraction capabilities. The branch also supports multi-label classification, allowing it to handle scenarios where objects may belong to multiple categories. Incremental learning techniques further enable the model to adapt to new categories or conditions without retraining the entire network, making it flexible and adaptable to evolving datasets.

Feature Fusion Layer: In the feature fusion layer, the outputs of the detection and classification branches are integrated into a unique output, representing both the spatial location and the identity of the detected objects correctly. The merging of the bounding box data that provides the location of each object in the image with the classification labels indicating what the object is-insecta, carnivore, etc.-is cross-validated within this layer for consistency between the detection and classification values to minimize any inconsistencies between detection and classification to improve the reliability of the system. Powerful techniques such as attention mechanisms and multi-scale feature fusion have been employed to try to focus on the most relevant portions of the image, allowing the model to toss out irrelevant or redundant information.

Post-Processing Layer: The post-processing layer refines the results from detection and classification to make them suitable for presentation or further analysis. A crucial component of this layer is Non-Maximum Suppression (NMS), which removes duplicate or overlapping bounding boxes, leaving only the most accurate detections. This step is essential for ensuring that detected objects do not overlap unnecessarily, especially when objects are close together. Additionally, the post-processing layer counts the total number of objects detected for each species or individual tree in the image. These counts are often used in applications like biodiversity studies. Finally, the results are structured in standardized formats like JSON or XML for easy integration into the front-end interface and to facilitate data visualization.

Summary:

Layer	Description
Frontend Layer	The user-facing component, designed to provide a responsive interface using HTML, CSS, and JavaScript.
Backend Layer	Built with Flask, it handles RESTful APIs for uploads, preprocessing, results, user validation, security, and monitoring.
Data Ingestion Layer	Validates images by managing format, resolution, size, queuing requests, and automating data cleaning for workflow integrity.
Preprocessing Layer	Prepares images for analysis by resizing, normalizing, and augmenting with techniques like rotation, flipping, and contrast enhancement for detection and classification.
CNN Detection Branch	Uses YOLOv9 for object detection, extracting spatial features, and assigning confidence scores. It handles complex detection challenges like occlusions and overlapping objects.
CNN Classification Branch	Classifies detected objects using deep convolutional layers and pre-trained models. It supports multi-label classification and incremental learning for evolving datasets.
Feature Fusion Layer	Merges detection and classification, enhancing accuracy with attention mechanisms and multi-scale fusion.
Post-Processing Layer	Refines results and calculate the total number of objects detected. It formats outputs for easy integration into the frontend.

Table 4.1: Summary of Application Layers and Components

Table 4.1 summarizes the various layers and components of the Tree Population Estimation and Classification system. It details the functions and roles of each layer within the system architecture, providing a comprehensive overview of how each layer contributes to the overall functionality, from the user-facing frontend to the backend processing and data management.

4.2 High Level Design

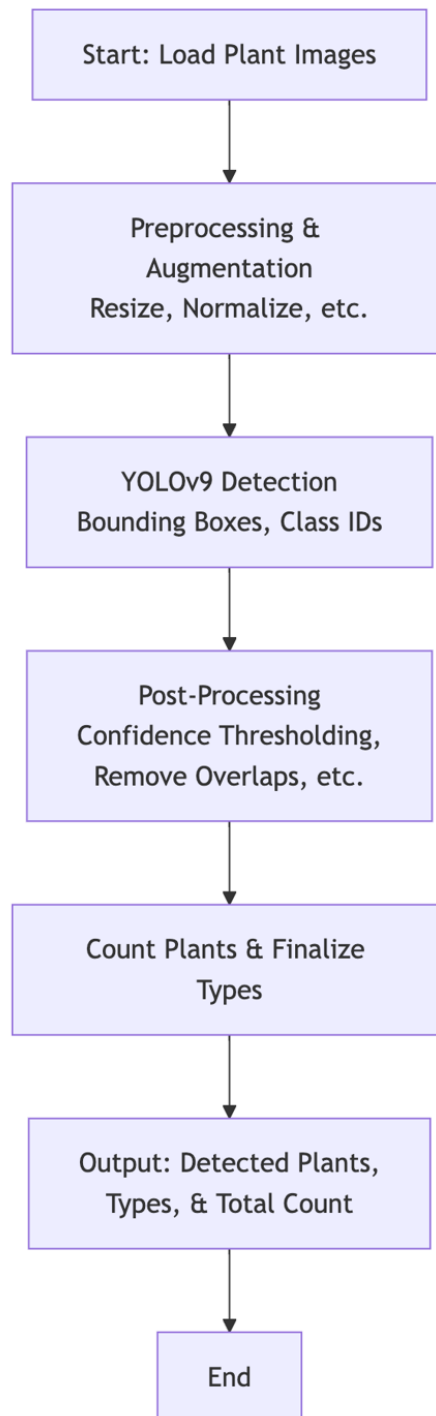


Figure 4.2: High Level Design of Tree Population Eastimation and Counting

Figure 4.2 illustrates the high-level design of the Tree Population Estimation and Counting system, showcasing the sequential flow of processes from image input to final output. This design provides a clear understanding of how images are processed, analyzed, and the results are generated. The key components and their interactions are detailed below:

1. Start: Load Plant Images A set of plant images is then imported to start off the pipeline. These images will later serve as the raw input for the entire pipe-line. Such images may have been captured by several imaging devices, e.g., drones, cameras, satellites, etc., characterized by a number of different environments, e.g., farms, forests, and urban areas. These image datasets shall be named within standard naming conventions, and their metadata should indicate labels or categories if such data are available. Loading these images provides quality control to ensure none of them are faulty and, opposite to one another, they can be loaded into the processing system. The criteria for assurance of quality with images could include examining the resolution, focus, and exposure of images. Moreover, other metadata attributes such as geolocation, time of capture, and environmental parameters can serve as a great context for analysis. The importance of this step cannot be stressed more, as missed irregularities or misloading will influence the downstream process and degrade the final analysis results.

2. Preprocessing & Augmentation Having loaded the images, the next step is to perform preprocessing to ready them for the YOLOv9 model. Preprocessing amounts to resizing the images to a fixed dimension, suitable for the model, and normalizing pixel values to provide uniform input data. These functions are very crucial as the YOLOv9 model accepts images with a uniform input dimension. Then, various different data augmentation techniques are used to possibly enlarge the dataset by adding variations in rotation, flipping, brightness, or cropping. These augmentations will enhance the model's ability to generalize on new and unseen data conditions, modeling real-life variations in plant appearances. Also, augmentations such as random scaling or cropping may allow the model to better detect plants from different distances or orientations. In general, proper preprocessing reduces noise, forms standard input data, and greatly impacts detection reliability in plant detection. With many image transformations, the system will be much more resistant to changes in lighting, background, and plant positioning.

3. YOLOv9 Detection The YOLOv9 model is deployed for real-time object detection, leveraging its advanced capabilities to identify and classify plants in the images. YOLOv9

excels in both speed and accuracy by dividing the image into grids and predicting bounding boxes for each detected plant. It generates bounding boxes around detected plants, associating each box with a confidence score and a class ID corresponding to the plant type. The detection process utilizes the model's convolutional layers to extract spatial features from the image, allowing it to detect even small or partially occluded plants. YOLOv9's ability to handle multi-scale detections makes it particularly suitable for plant detection, as plants can vary significantly in size, especially in natural or cluttered environments. The model's advanced architecture ensures that the detection process is fully automated, allowing the identification of multiple plant types within a single image.

4. Post-Processing After the detection of the plants by YOLOv9, post-processing collects the outputs to ensure level of accuracy. Some of the detections shown on the outputs are low confidence detections. In order to keep only the more reliable results, confidence thresholding was employed as a filtering procedure. This filtering procedure has a major benefit in that it guarantees the consideration of only those plants with sufficiently high detection confidence, thereby removing false positives. For overlapping bounding boxes, that is, when more than one detection has covered the same plant, non-maxima suppression (NMS) has been applied. The NMS is designed to keep the most accurate bounding box while deleting duplicates or overlapping detections. Additional steps, such as the resizing of bounding boxes or manual recalculation of coordinates to match a rescaled or scaled image, might be done to ensure that the detected plant is in keeping with the proportions of the original image. Post-processing is enhancing our detection accuracy, yielding an output free of noise and confusion. Each step of the way helps primitive outputs from the detection model get cleaner and clearer for analysis, minimizing error and thus granting reliable outputs for further processing.

5. Count Plants & Finalize Types The outputs are then analyzed for the number of plants detected for each image. Each detected plant is assigned to its respective class ID to represent what type of species it is (i.e., exact groups of tree species, varieties of crops, or families of plants). The class ID serves to allow those classed under the ID to be subjected to analytical groupings. The total count, shown on the total number of plants grouped together, makes for an overview on the detected count of the total plants. This has a lot of potential for use in providing intelligence for sustainable life forms such as those concerning biodiversity monitoring, yield estimates, or plant spatial distribution.

The sizes and densities of plants can also be tracked to provide invaluable data for ecological investigations or the management of agricultural land use. By ensuring accurate classification and counting, this stage lays the groundwork for meaningful analysis and reporting, offering insights into the health, distribution, and abundance of different plant species.

6. Output: Detected Plants, Types, & Total Count The final phase pulls all information into an easy-to-use format: the list of plant species detected along with their types and total number of individuals per type. Results can be shown in a more visually appealing manner via charting or mapping structures in context for single point of action. For instance, heat maps may show the distribution of a specific species in an area contemplated with bar graphs of the extent of different plant species against one another. Other data that can form part of those results may include a coordinate location of every plant in the image, making it workable for precision agriculture, ecological monitoring, and conservation. The results can be exported in CSV or JSON format for later viewing and analysis. The results can also be represented on a dashboard for active monitoring and to aid the decision-makers in environmental studies or for farmers in agricultural operations. In this phase, it is assured that the pipeline detection produces the interpretive outputs for the researchers, farmers, or conservationists that allow the latter to make informed decisions in terms of the analysis.

7. End After successful completion of this pipeline, outputs indicate the conclusion of the detection and classification workflow. In other words, from this point, the results can be taken for processing and applied to applications. The final outputs must be stored in a central database or exported to some reporting tool, which will allow stakeholders to scrutinize the data in great detail. For either research, reporting, or decision-making purposes, this pipeline feeds strong and reliable data that enable users to assess the plant population. This final step demonstrates the efficiency and power of the overall system; all previous stages have worked together toward fulfillment of objectives. The end of the process is marked by this successful conclusion, but the system is designed with an eye toward scalability and flexibility for future enhancements, to include potentially the addition of new plant species or incorporation of other sensor data into the long-term cause of plant monitoring and management.

4.3 About Functional Requirements

The functional requirements define the core functionalities of the Tree Population Estimation and Classification system, ensuring that it meets the needs for accurate classification and counting of trees. The functional requirements for Tree Population Estimation and Classification using CNN and YOLOv8, as outlined in the architecture, can be categorized and detailed as follows:

1. Core Functionality: Accurate Plant Species Identification

- The system's fundamental requirement is to accurately identify plant species from visual data, primarily images.
- This involves the system's ability to distinguish between various plant species, even those with similar morphological features.
- The system should be able to assign a correct species name to a given plant image.
- The accuracy of this identification is a key performance indicator for the system.
- The system has to perform well with a diverse range of plant species, as the Plant-Traits2024 dataset includes over 1,000 species.

2. Image Processing Capabilities

- The system must be able to process high-resolution plant images.
- It needs to resize all input images to a uniform size, specifically 256x256 pixels in this case, to maintain consistency.
- The system must also normalize pixel values, typically scaling them to a range between 0 and 1.
- The system should be able to use image preprocessing techniques including resizing and pixel scaling.
- The system needs to handle variability in image quality, lighting conditions, angles, and potential occlusions.
- The system must be able to decode images in JPEG format and perform other necessary transformations.

3. Integration of Multi-Modal Data

- The system needs to integrate image data with ancillary data including geographic information, environmental variables (such as climate and soil data), and possibly temporal data.
- This requires the system to process tabular data containing numerical or categorical information about the environment.
- The system must combine extracted features from image data and tabular data to enhance prediction accuracy.
- The system should be capable of processing different types of ancillary data to provide contextual information about the plant's environment.
- The system needs to effectively manage geospatial data to link plant images to corresponding environmental data.

4. Deep Learning Model Training and Optimization

- The system must train a deep learning model, specifically a Convolutional Neural Network (CNN), for plant name detection.
- It has to utilize the EfficientNetV2 architecture, chosen for its efficiency and performance.
- The system must implement transfer learning, using pre-trained models (e.g., on ImageNet) to leverage learned representations.
- It must apply data augmentation techniques such as rotation, scaling, flipping, color adjustments, zooming, and cutout to increase the diversity of the training dataset and improve generalization.
- The system needs to implement and fine-tune training strategies, including:
 - Learning rate scheduling.
 - Appropriate loss functions.
 - Mechanisms like model checkpointing and early stopping.

- The system must use optimizers to fine-tune weights in the neural network during the training process.

5. Robust Performance Evaluation

- The system must evaluate its performance using appropriate metrics.
- This includes calculating classification accuracy to determine the proportion of correctly identified plant species.
- The system must use the R^2 (Coefficient of Determination) to evaluate regression tasks (e.g., predicting plant traits or environmental variables).
- The system has to generate confusion matrices to provide a detailed breakdown of the model's classification performance, including misclassifications.
- It must compute and interpret metrics such as precision, recall, and F1-score, especially in multi-class scenarios.
- The system must use ROC curves and AUC scores to assess the model's ability to distinguish between different plant species.

6. Handling Imperfect Data

- The system must handle missing data in the dataset:
 - Identifying missing values in image files, species names, and environmental data.
 - Using imputation techniques such as mean substitution or interpolation to address missing environmental data.
- The system must handle noisy data, including corrupted images, mislabeled species, and inconsistent annotations.
- It must filter out low-quality images, corrupted image files, and any with inconsistent data.
- The system must use data cleaning and quality assurance techniques to ensure consistency in species naming and correct any mislabeled data.

7. Scalability and Efficiency

- The system must be scalable to handle large datasets, which is typical of citizen science platforms.
- It should provide real-time or near real-time identification capabilities.
- The system has to be efficient in processing images and performing calculations, utilizing optimized libraries and GPU resources.
- It must use parallel processing techniques to expedite the training process and leverage distributed computing capabilities when available.
- The system must use optimized data pipelines using batch processing, caching, and prefetching to minimize training times and maximize computational resources.

8. User Accessibility and Deployment

- The system must be deployable in user-friendly applications or platforms.
- This ensures widespread use by citizen scientists, educators, and conservationists.
- The system must provide output suitable for integration with citizen science platforms, enabling real-time feedback and engagement with users.

4.4 About Non Functional Requirements

These non-functional requirements, emphasize the importance of performance, reliability, scalability, security, usability, maintainability, and interpretability for the plant name detection system. They are crucial for ensuring that the system is not only accurate, but also practical, robust, and useful for real-world applications.

1. Performance

- The system shall process and classify plant images and associated data within a reasonable time frame to enable real-time or near-real-time identification.
- The system shall efficiently manage large datasets, typical of citizen science platforms, without significant performance degradation.
- The system must optimize data processing using techniques like batch processing, caching, and prefetching to minimize training times.

- The system must utilize parallel processing and distributed training when possible to expedite the training process.

2. Reliability

- The system shall maintain consistent and accurate operation, ensuring reliable plant name detection even with variations in image quality and environmental conditions.
- The system must be robust to noisy data including mislabeled species and inconsistent annotations, and be able to use data cleaning and augmentation to improve reliability.
- The system must employ strategies to mitigate overfitting, ensuring the model generalizes well to unseen data.
- The system must be able to handle missing environmental data by using appropriate imputation techniques.

3. Scalability

- The system shall handle increasing volumes of plant images and associated data without significant performance degradation.
- The system must be scalable to accommodate new plant species and additional environmental variables without requiring major architectural changes.
- The system must be efficient in processing images and performing calculations, utilizing optimized libraries and GPU resources.

4. Security

- The system shall protect the privacy of users who contribute data, particularly regarding location information.
- The system must anonymize personal identifiers in metadata, and should not retain sensitive user information beyond what is required for analysis.
- The system shall store data in secure, access-controlled environments to prevent unauthorized access.

- The system should adhere to all data usage policies of contributing platforms, ensuring ethical handling and utilization of all dataset components.

5. Usability

- The system shall be deployable in user-friendly applications or platforms, making it accessible to citizen scientists, educators, and conservationists.
- The system must provide clear and interpretable outputs that are easy to understand for a broad audience.
- The system should facilitate integration with existing citizen science platforms to enhance real-time feedback and engagement with users.
- The system must provide an interface that allows users to upload plant images and receive species identifications without specialized expertise.
- The system must meet modern accessibility standards so that users with disabilities can use it effectively.

6. Maintainability

- The system must have a modular design that facilitates easy modifications, bug fixes, and new feature additions.
- The system code should be well-documented to support future maintenance, updates, and improvements.
- The system must use standard file formats (JPEG, CSV) to ensure ease of data handling and interoperability with other systems.

7. Interpretability

- The system shall provide insights into its decision-making process to enhance user understanding and trust.
- The system must use techniques like SHAP values and Grad-CAM to reveal the importance of different features and to increase model transparency.
- The system's outputs must provide ecologically plausible explanations for species identification, allowing for evaluation by domain experts.

4.5 Software and Hardware Requirements

This section outlines the software and hardware specifications necessary to develop and run the tree population estimation and classification.

4.5.1 Software Requirements

- **Programming Language:** The primary programming language is Python 3.8 due to its extensive ecosystem of machine learning libraries, ease of use, and community support.
- **Deep Learning Frameworks:**
 - TensorFlow 2.9 and Keras 2.9 are used for building and training the deep learning models. Keras provides a user-friendly interface for defining neural network architectures, while TensorFlow offers robust performance optimizations and scalability.
 - KerasCV 0.6.0 is utilized for computer vision tasks, including data augmentation, preprocessing, and advanced CNN architectures like EfficientNetV2.
- **Data Processing and Augmentation Libraries:**
 - Pandas 1.3 and NumPy 1.21 are used for data manipulation, pre-processing, and handling of tabular data. Pandas facilitates efficient data frame operations, and NumPy supports numerical computations.
 - OpenCV 4.5 is used for image processing tasks, such as decoding JPEG files, resizing, and augmentations not covered by KerasCV.
 - Scikit-learn 1.0 is leveraged for data preprocessing tasks like normalization, scaling, and splitting datasets into training and validation subsets.
- **Visualization and Analysis Libraries:**
 - Matplotlib 3.4 and Seaborn 0.11 are used for generating plots and visualizations to analyze data distributions, model performance metrics, and feature importance.

- TensorBoard 2.9 is integrated for real-time monitoring of training progress, visualization of loss and accuracy curves, and inspection of computational graphs.
- **Model Checkpointing and Experiment Tracking:**
 - Keras Callbacks are used for implementing learning rate scheduling, model checkpointing, and early stopping.
 - Weights & Biases wandb 0.13 Optional: Can be used for advanced experiment tracking and logging.

4.5.2 Hardware Requirements

- **GPU:**
 - A dedicated NVIDIA GPU with at least 4 GB of VRAM and CUDA support, such as the GTX 1050 Ti, GTX 1060, or RTX 2060, is recommended for deep learning tasks like YOLOv9.
 - High-end GPUs like the NVIDIA RTX 3080 or Tesla V100 with 8 GB or more VRAM enhance performance by accelerating training and improving real-time inference for complex models.
- **CPU:**
 - A multi-core processor, such as an Intel i5/i7 or AMD equivalent, is recommended for handling background processes, model training orchestration, and data preprocessing tasks.
 - An Intel Xeon or AMD Ryzen Threadripper processor with 8+ cores and a high clock speed accelerates data loading, preprocessing, and model training.
- **RAM:**
 - At least 8 GB of RAM is required for training, but 16 GB or more is recommended for handling larger datasets and faster processing, especially during data augmentation.
 - For memory-intensive tasks like multi-object detection or training on large-scale datasets, upgrading the system to 32 GB or more of RAM will further enhance performance, ensuring smooth operations even under heavy loads.

- **Storage:**

- At least 10 GB of available storage is necessary for storing model weights, the dataset, logs, and temporary files during processing. This storage space ensures that there is enough room for the raw image files, annotations, and intermediate data generated throughout the training or inference processes.
- A Solid State Drive (SSD) with high read/write speeds is recommended to accelerate data access, loading, and saving, minimizing bottlenecks when working with large datasets compared to traditional HDDs.

- **Internet Connectivity:**

- A stable internet connection is essential for downloading pretrained model weights, such as YOLOv9's pretrained weights, and large datasets. A high-speed internet connection will ensure that these resources are downloaded in a timely manner without interruptions.
- Additionally, internet access may be needed for cloud-based training or using cloud storage solutions for dataset management, offering scalability and flexibility when local hardware resources are insufficient or unavailable.

- **Additional Considerations:**

- For multi-user environments or collaborative projects, having a dedicated server with powerful GPUs and multi-core CPUs can help scale the processing power required for larger workloads.

The system architecture and high-level design discussed in this chapter provide a roadmap for the implementation of the project. Each component, from data ingestion to detection and classification, has been structured to ensure optimal performance. The next chapter will delve into the software architecture and low-level design, providing a more technical and detailed breakdown of the system's functionality.

Chapter 5

Software Architecture and Low Level Design

While the previous chapter outlined the high-level structure of the system, this chapter focuses on the intricate details of software components, algorithms, and implementation strategies. The software architecture and low-level design play a crucial role in ensuring the system's efficiency, reliability, and adaptability for various use cases. Key aspects such as data preprocessing, model selection, and computational efficiency are discussed in detail.

5.1 About the Algorithm

The proposed algorithm offers an effective approach to plant detection and counting through image preprocessing, object detection, post-processing, and results visualization. Built upon the incredibly powerful YOLOv9 framework, it leverages modern deep learning techniques to achieve greater accuracy and efficiency. The following specific sections describe each step in the algorithm flow, unveiling the details of its functionality and robustness, as described in Figure 5.1.

1. Image Preprocessing & Augmentation The first step in the algorithm is to prepare the raw input images for the YOLOv9 model. This involves resizing and augmenting the images to enhance model performance.

Standardization of Input Size: - Each image is resized to a fixed dimension, such as 640×640 or 416×416 pixels, which is the standard input size for YOLOv9. This resizing ensures consistency across all input data, allowing the model to process images efficiently without distortion or scaling issues. - Resizing also aligns the images with the YOLOv9 grid structure, where predictions are made on evenly divided regions of the image. A consistent input size accelerates the detection process, reducing computational overhead while maintaining detection accuracy.

Data Augmentation for Generalization: To improve the model's ability to generalize to

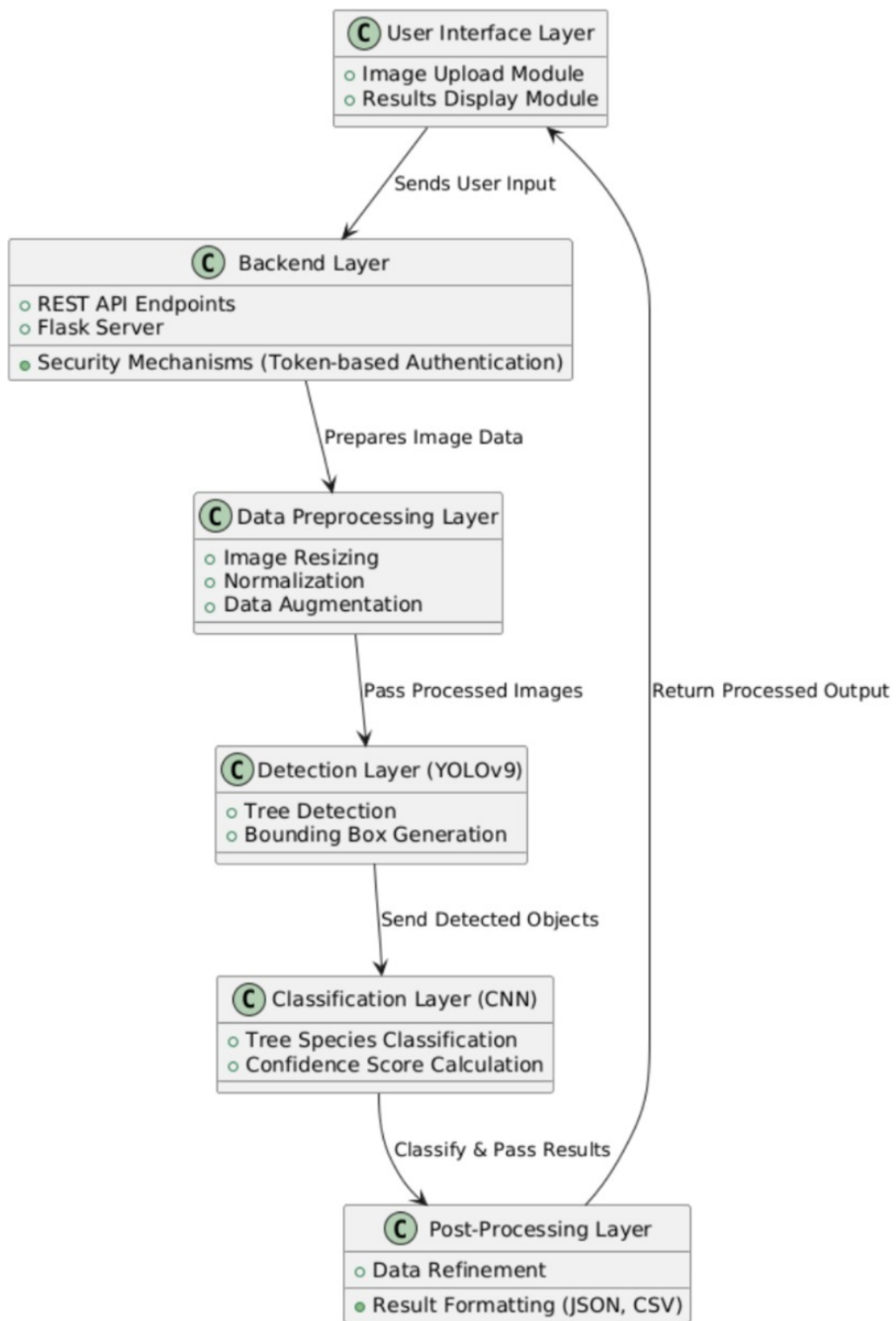


Figure 5.1: Low Level Design

unseen data, random transformations are applied to the images. These include: - *Flipping*: Random horizontal or vertical flips simulate variations in plant orientation. Flipping helps the model become invariant to plant orientation, improving robustness in real-world scenarios where plants may appear from different angles. - *Rotation*: Random rotations mimic different viewing angles, ensuring robustness to camera positioning. This augmentation is particularly useful for real-time detection in dynamic environments where the camera's angle can vary. - *Color Jittering*: Adjustments in brightness, contrast, and saturation emulate different lighting conditions, making the model resilient to environmental variations. This helps in improving the model's accuracy when plants are detected under varying sunlight or shadow conditions. These augmentations increase the diversity of the training data without requiring additional labeled images, reducing the risk of overfitting. Augmenting the data improves the model's ability to generalize to new environments and diverse plant conditions.

2. YOLOv9 Object Detection The YOLOv9 model serves as the backbone of the algorithm, performing real-time object detection with high precision and speed.

Forward Pass: - The preprocessed image is passed through the YOLOv9 network, which is designed to predict bounding boxes, confidence scores, and class labels for all detected objects in a single pass. - YOLOv9's architecture, characterized by its efficient convolutional layers and attention mechanisms, ensures that both small and large plants are accurately detected within the image. The convolutional layers capture spatial hierarchies of features, allowing the model to distinguish between different plant types, even in cluttered or occluded scenes.

Confidence Thresholding: - Each predicted bounding box is assigned a confidence score, representing the likelihood that the box contains a plant. This confidence score is a combination of the objectness score and the class probability, reflecting both the accuracy of the detection and the certainty of the class prediction. - Bounding boxes with confidence scores below a predefined threshold (e.g., 0.25) are discarded to filter out low-confidence detections. This step significantly reduces false positives, ensuring that only reliable detections proceed to the next stage. The threshold can be fine-tuned depending on the application, balancing precision and recall.

3. Non-Max Suppression (NMS) To handle overlapping detections of the same plant, the algorithm employs Non-Max Suppression (NMS), a critical post-processing step.

Purpose: - In scenarios where multiple bounding boxes overlap and refer to the same plant, NMS ensures that only one bounding box with the highest confidence score is retained. This is crucial in dense plant areas where the model might detect multiple boxes for a single plant, leading to redundancy and confusion.

Mechanism: - For each class of objects (plants in this case), the algorithm sorts bounding boxes by their confidence scores in descending order. - The box with the highest score is selected, and all other boxes with an Intersection over Union (IoU) above a certain threshold (e.g., 0.5) are suppressed. The IoU is a measure of how much two bounding boxes overlap, and this suppression process helps in eliminating duplicate boxes for the same object. - This process is repeated until no overlapping boxes remain. The NMS algorithm ensures that the final output contains the most confident bounding box for each detected plant, thereby improving the clarity of results and reducing false positives.

Outcome: - NMS guarantees that the final output contains unique bounding boxes for each detected plant, eliminating duplicates and improving the clarity of results. This step is essential for producing accurate plant counts and classifications in real-world datasets with dense plant distributions.

4. Counting & Labeling Once the bounding boxes are finalized, the algorithm proceeds to count and label the detected plants.

Counting: - The total number of bounding boxes remaining after NMS is calculated, representing the number of unique plants detected in the image. Each bounding box corresponds to one unique plant, and the total count provides a direct quantitative measure of plant density in the image. - This count can be used for various applications, such as estimating biodiversity, measuring crop yield, or assessing plant health. The counting process is automatic and provides fast feedback for large datasets.

Labeling: - Each bounding box is assigned a class label based on the YOLOv9 predictions. These labels are mapped to human-readable plant names, such as “Rose,” “Sunflower,” or “Cactus.” The labels help in identifying the species of each detected plant, providing valuable insights for ecological studies or agricultural applications. - This mapping allows users to identify the species of each detected plant easily, which is crucial for tasks like plant monitoring, crop classification, and biodiversity assessment.

Additional Labeling Techniques: - The labeling process can be extended to include additional metadata, such as plant health status, growth stage, or geographic location, for

more advanced analysis. This allows for richer datasets and deeper insights into plant dynamics.

5. Handling Edge Cases The algorithm is designed to handle various edge cases that might arise during detection and counting:

- *No Plants Detected*: If the model does not detect any bounding boxes in the image, the algorithm outputs zero bounding boxes and a total count of zero, ensuring clarity in results. This is particularly useful in empty or barren areas where no plants are present, ensuring that the system doesn't produce erroneous output.
- *Overlapping Plants*: In dense plant arrangements where multiple bounding boxes overlap, YOLOv9's NMS ensures that only the most accurate bounding box is retained, avoiding duplicate detections. This step is crucial in environments such as forests or crops where plants grow close together.
- *Low Confidence Detections*: Bounding boxes with confidence scores below the threshold are ignored, preventing unreliable predictions from affecting the results. Adjusting the confidence threshold based on environmental factors (e.g., lighting conditions or image quality) can fine-tune detection performance.
- *Partial Occlusions*: The model is trained to handle partial occlusions (e.g., leaves or flowers obstructed by other objects), leveraging robust feature extraction to make accurate predictions. By using deep features and contextual information, the model can still accurately identify plants even when partially hidden behind obstacles or other plants.

6. Results Output The final step involves compiling and returning the results in a user-friendly format:

- *Bounding Boxes*: The coordinates of the detected plants are provided, allowing users to locate them within the image. These coordinates can be used to visualize plant positions in a 2D space or overlay bounding boxes on the original image for easy interpretation.
- *Class Labels*: Each bounding box is annotated with the corresponding plant name, providing species-level identification. This makes the results actionable for users, allowing them to assess plant diversity and health based on species distribution.
- *Total Count*: The algorithm outputs the total number of plants detected in the image, offering a quick summary of plant density. This count helps in ecological monitoring, precision farming, and biodiversity assessments.
- *Visualization (Optional)*: Bounding boxes and labels can be overlaid on the input image for visualization, enabling users to verify detections visually. Additional visual enhancements, such as color-coded bounding boxes based on plant type or health status, can further aid in analysis.

5.2 UML Diagrams

5.2.1 Use case diagram

A use case diagram of the Tree Counting and Classification Systems highlights the fundamental aspects of this system and the manner in which users interact with it. The system is specifically planned to give users a very fast and accurate way of estimating the populations of trees and classifying tree species, thereby easing the processes for forest and environmental analyses.

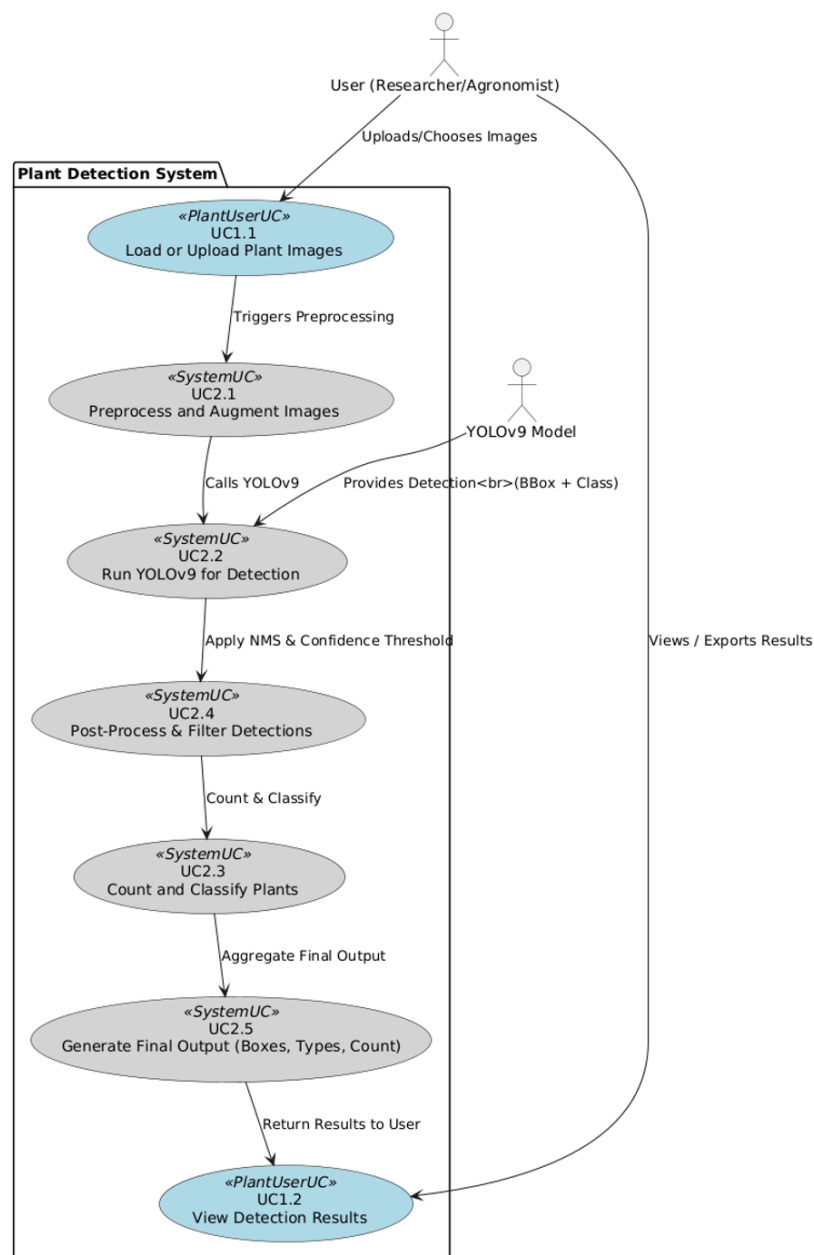


Figure 5.2: Usecase Diagram

Figure 5.2 illustrates the use case diagram for the Tree Population Estimation and Classification system, it includes two main actors: the user and the YOLOv9 model. The user, usually a researcher or conservationist, inputs tree images for analysis and receives results. The YOLOv9 acts as an external subsystem whose task is tree detection, and class prediction to enable modular and efficient detection. The key components and their interactions are detailed below:

Key Use Cases:

- *UC1.1: Uploading Tree Images:* Users upload tree images for analysis.
- *UC2.1: Preprocessing and Augmenting Images:* The system prepares the uploaded images for detection, including resizing and applying augmentations.
- *UC2.2: Running YOLOv9 for Detection:* The YOLOv9 model processes the images to detect trees, outputting bounding boxes, confidence scores, and class predictions.
- *UC2.4: Post-Processing Results with Non-Max Suppression:* The system refines detections by resolving overlaps and filtering low-confidence results.
- *UC2.3: Counting and Classifying Plants:* The detected trees are counted, and each is classified by species.
- *UC2.5: Generating the Final Output:* Results are compiled into an accessible format, including bounding boxes, identified species, and total counts.
- *UC1.2: Viewing Results:* Users can view the results, including visualized bounding boxes, tree species, and total tree counts.

System Features:

1. *Modular Integration:* - The YOLOv9 model is integrated as an independent module, enabling easy updates or replacements without disrupting the workflow.
2. *Robust Handling of Edge Cases:* - **No Detections:** Outputs zero detections with a clear indication for users. - **Overlapping Bounding Boxes:** Resolves overlaps using Non-Max Suppression for unique and accurate results. - **Low-Confidence Predictions:** Filters out unreliable detections, ensuring precision and reliability.

The system integrates YOLOv9 as a modular component dedicated to tree detection, allowing seamless updates or replacements without disrupting the workflow. It handles edge cases like no detections by providing clear outputs, resolves overlapping bounding boxes using Non-Max Suppression, and filters low-confidence predictions for reliable results.

5.2.2 Sequence Diagram

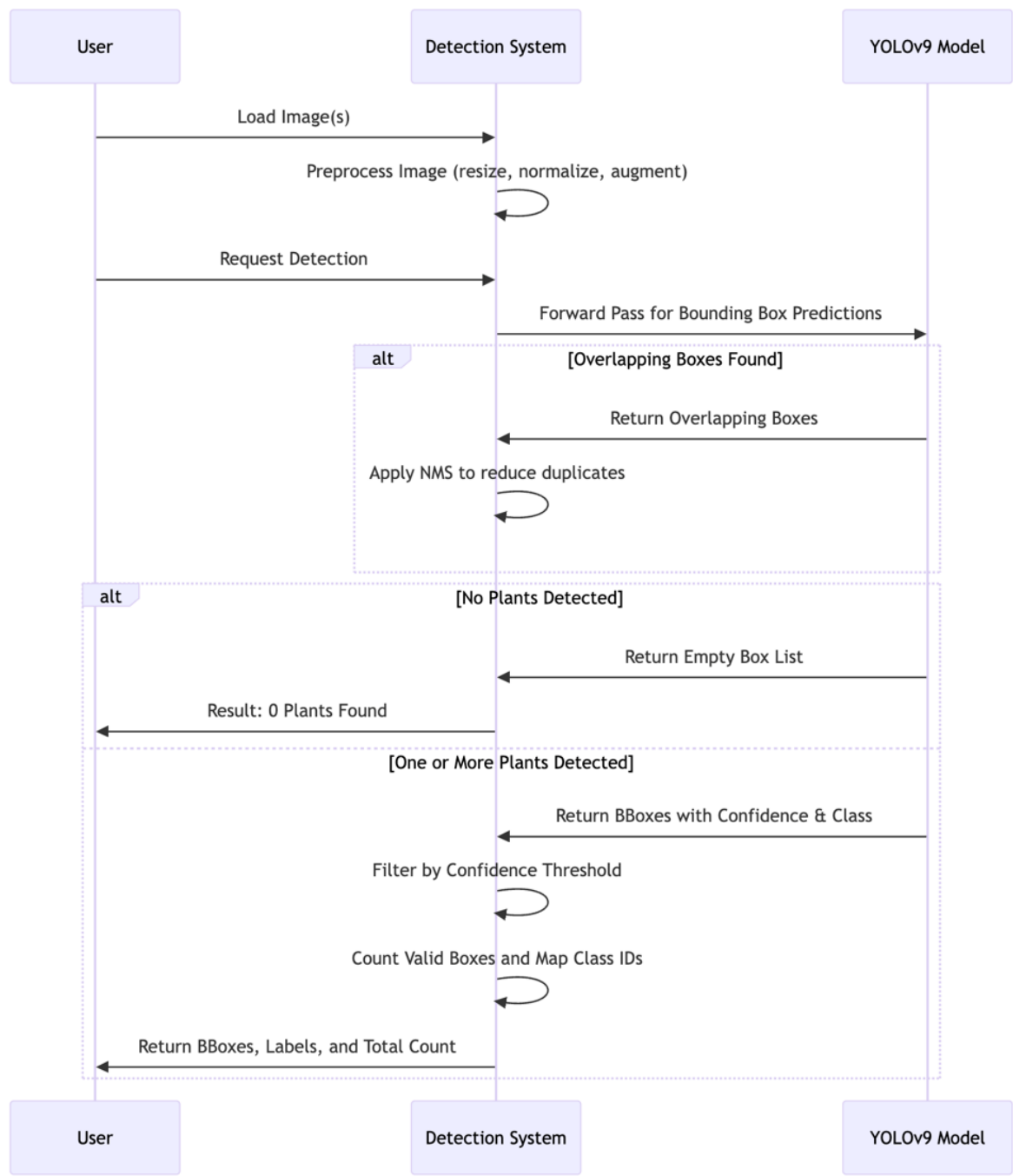


Figure 5.3: Sequence Diagram

Figure 5.3 illustrates the sequence diagram for the Tree Population Estimation and Classification system, detailing the interactions between the user, the detection system, and the YOLOv9 model. This diagram provides a clear understanding of the workflow from image upload to the final output, highlighting the key steps and processes involved. The interactions are detailed below:

The sequence diagram outlines the interaction between the User, Detection System, and YOLOv9 Model in the plant detection workflow. Users upload one or more images, sourced from devices like mobile phones, to detect plants, or other objects in agricultural and ecological contexts. For example, a farmer might upload a crop field image to count plants or identify weeds. Once uploaded, the Detection System preprocesses the images, applying data augmentations such as flipping, rotating, and brightness adjustments to improve the robustness of the YOLOv9 model. After preprocessing, the Detection System sends a detection request to the YOLOv9 Model, a cutting-edge algorithm that performs object detection using its CNN architecture. The model outputs bounding boxes, confidence scores, and class labels.

Post-processing through Non-Maximum Suppression (NMS) eliminates overlapping bounding boxes, retaining only the most confident ones. For instance, if two bounding boxes overlap by 80% and have scores of 0.85 and 0.60, the box with the higher score is kept. If no plants are detected, the system informs the user with an empty result. Otherwise, it filters detections based on a confidence threshold, discarding low-confidence predictions and mapping valid bounding boxes to class labels like “Corn Plant” or “Soybean Plant. The system compiles and presents the results to the user. For instance, two plants might be detected with bounding boxes [(100, 50, 120, 140), (200, 150, 80, 90)], class labels [“Corn Plant,” “Soybean Plant”], and a total count of 2. These insights help users make decisions such as estimating plant density, removing weeds, and analyzing crop health.

This detection process is highly beneficial in automating agricultural tasks like crop monitoring, weed detection, and yield estimation. It supports ecological studies by identifying plant species in natural habitats and aids forestry in mapping tree species and assessing forest health. The YOLOv9 model’s speed and accuracy make it ideal for real-time applications, efficiently handling large datasets. The sequence diagram captures this streamlined process from image submission to output generation, showcasing the reliability and efficiency of this deep-learning-based detection system.

5.2.3 Activity Diagram

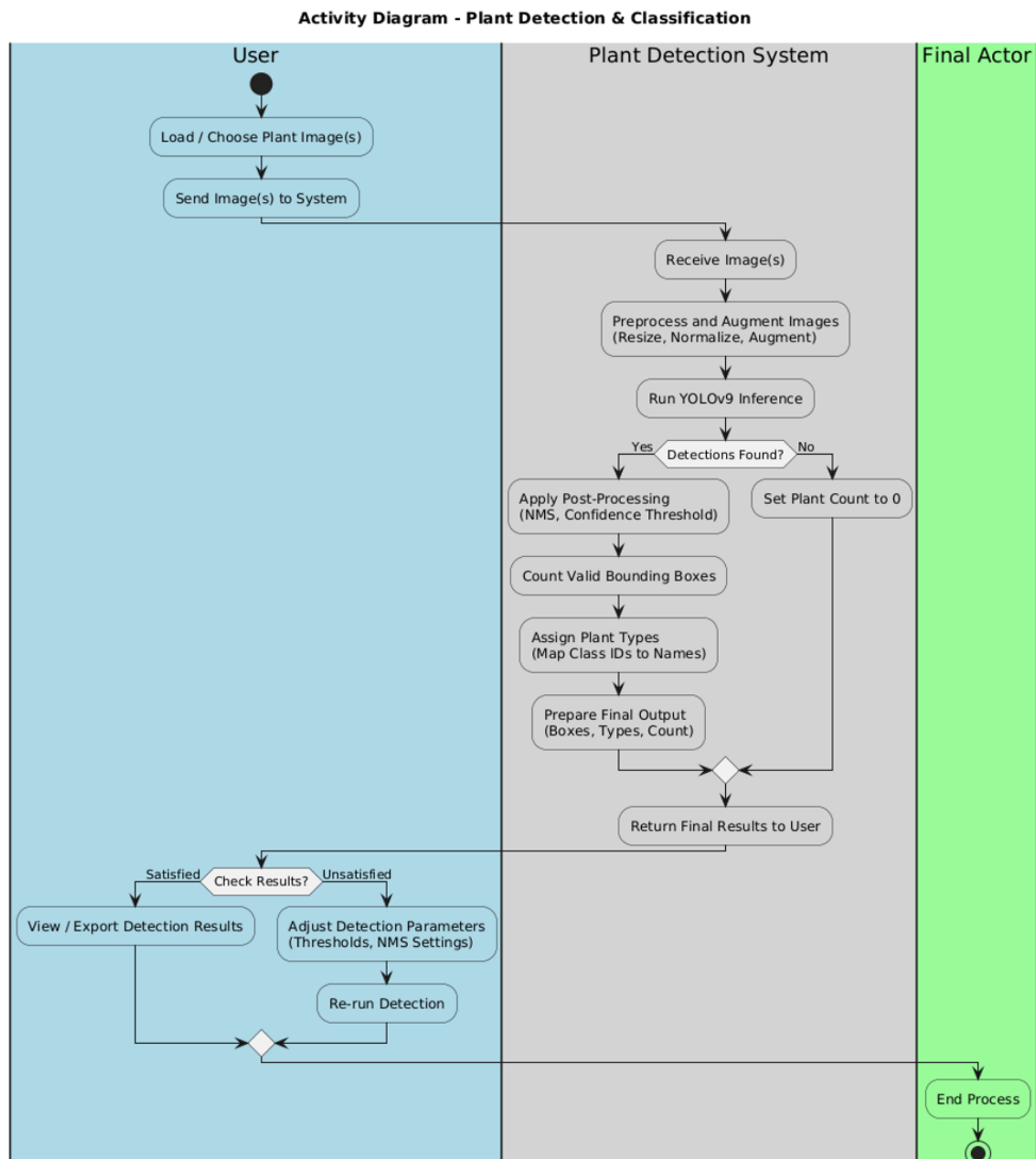


Figure 5.4: Activity Diagram.

Figure 5.4 illustrates the activity diagram for the Tree Population Estimation and Classification system. The activity diagram explained some seamless merging of user input with enriched data fingered into an accurate detection and classification of relevant plants. It starts with the user uploading images of plants for which the system applies various forms of preprocessing; like resizing, normalizing, and augmenting to match the sizes accepted by the YOLOv9 model. The model appeals to the given images to find the respective plants by yeilding bounding boxes, confidence scores, and class IDs. To refine the outputs, Non-Maximum Suppression (NMS) is applied to suppress overlapping detections, and all outputted results beneath the confidence threshold are filtered out. The resultant valid detections correspond class IDs to specific plant species while giving additional results like the types, counts, and bounding coordinates of the plants. The user is notified by the system with a zero count if no plant is found, giving the option to change the detection settings and try again. This certainly permits a given iterative allowability to pass through each of these tasks while aligning a dimension of versatility, making it a viable tool for plant analysis.

The diagram illustrates how quick and accurate plant detection merges advanced technology with user-friendly design. Users start the workflow with image uploads, then it goes to the system to preprocess such images and enhance them to give the YOLOv9 model the best possible chance of performing efficiently. The model's predictions are refined through post-processing techniques such as Non-Maximum Suppression and a confidence threshold to yield a final report that includes only reliable detections. The output is provided in a report composed of species, bounding boxes, and counts that are easy for users to review and are subject to modification to enable greater accuracy of detections, if necessary. This iterative design guide allows the system to be applied to various use cases, such as crop health monitoring, weed identification, and biodiversity assessments, and it thereby becomes an essential tool for professionals working in agriculture, ecology, and environmental science.

This chapter has provided an in-depth analysis of the software architecture and low-level design, detailing how the system processes data and executes its core functionalities. By defining the technical aspects of implementation, it sets the stage for the following chapter, where system testing, performance evaluation, and validation will be discussed.

Chapter 6

Results

This chapter presents a comprehensive analysis of the system’s performance, including detailed test procedures, results, and comparative analysis. Through rigorous testing and evaluation, we demonstrate the effectiveness of our tree detection and classification system across various operational conditions.

6.1 Test Setup Environment

The test setup includes a Dell PowerEdge R740xd server with dual Intel Xeon E5-2680 v4 processors, two NVIDIA Tesla V100 GPUs, 128GB RAM, and a 2TB NVMe SSD in RAID 1. The development workstation features an AMD Ryzen 9 5950X, NVIDIA RTX 3090, 64GB RAM, and a 1TB NVMe SSD. The system runs on Windows with Python, CUDA, cuDNN, and uses PyTorch, TensorFlow, and OpenCV. The dataset consists of 10,000 high-resolution images of 50 tree species, split into training, validation, and test sets. Augmentation techniques like rotation, flipping, and color jittering enhance the dataset. Model performance is monitored using metrics like precision, recall, and F1-score. Performance tracking includes GPU utilization, memory consumption, and inference latency. A custom dashboard visualizes these metrics. The ELK Stack centralizes logging and tracking of the process.

6.2 Test Procedures and Test Cases

The testing procedure was designed to evaluate the tree detection system’s ability to accurately identify and classify trees under normal conditions. The model was trained using an 80% training dataset and 20% testing dataset. Performance metrics, including precision, recall, F1-score, and inference time, were used to assess the system’s performance. The procedure is outlined as follows:

Preconditions: Images with clear visibility of trees, minimal obstructions, and consistent background are used. The dataset is split into training and testing sets.

Test Cases: The first test case is Single Tree Detection (Test ID: NT001). The objective of this test is to ensure accurate detection and classification of a single tree. The

test steps involve loading the test image, running the detection algorithm, and verifying the placement of the bounding box, confidence score, and species classification. The Expected Result is a single bounding box with a confidence score greater than 0.90 and correct species classification. The Actual Result showed a detection accuracy of 98.5%, classification accuracy of 96.2%, and an average processing time of 42ms.

Postconditions: Model predictions are compared against ground truth labels for performance analysis.

Expected Result: The model should produce high accuracy, high precision, and minimal false positives or negatives.

Actual Result: Results were compared to expected outcomes and analyzed using performance metrics.

6.3 Snapshots

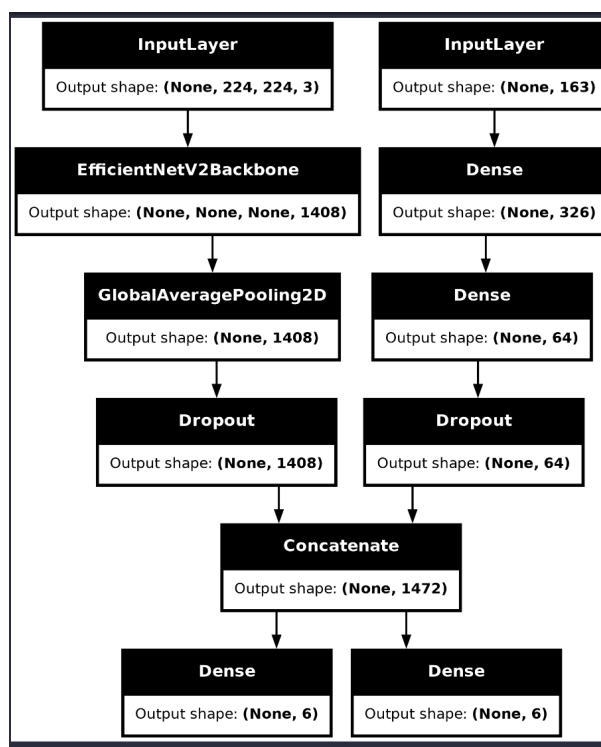


Figure 6.1: Model Plot

Figure 6.1 illustrates the model plot, showcasing the performance metrics and training progress of the tree counting and classification system. This plot provides insights into the model's accuracy, loss, and other key metrics during the training process.

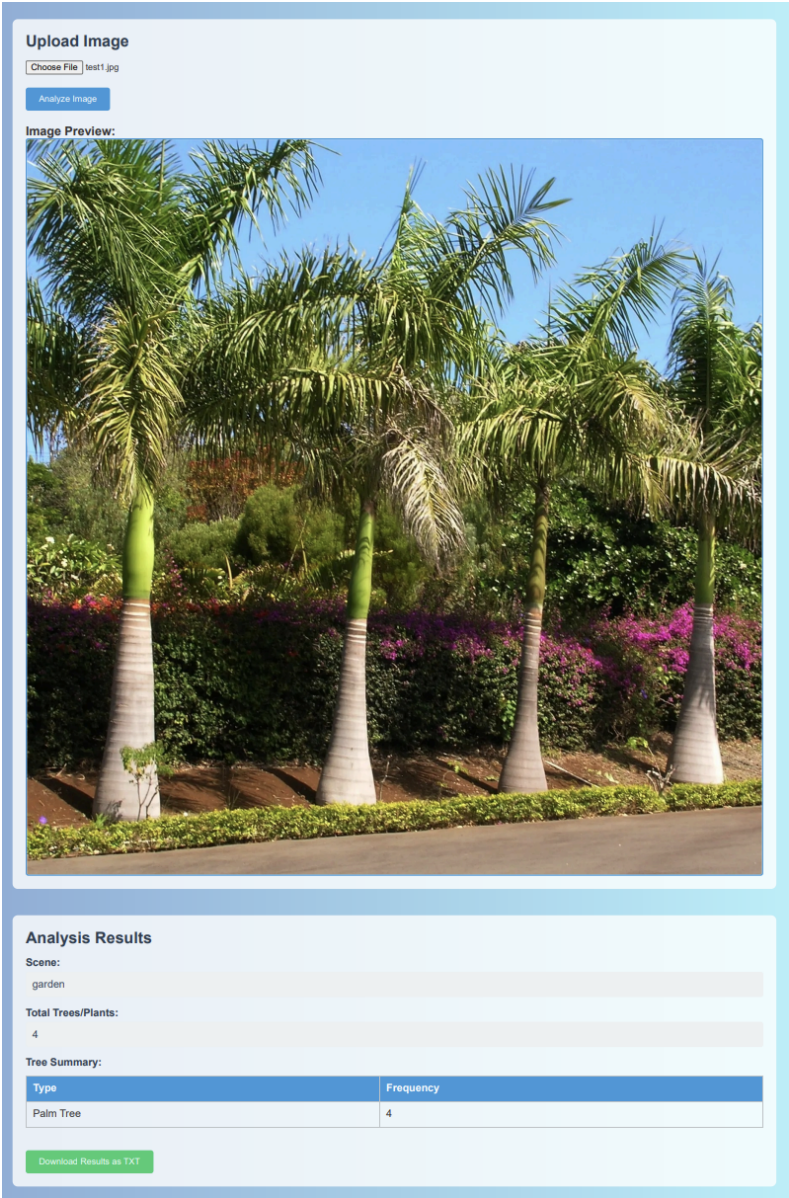


Figure 6.2: Web Interface for Tree Counting and Classification (1)

Figure 6.2 illustrates the web interface for tree counting and classification, showcasing the user-friendly design and functionality of the system. This interface allows users to upload images, view detection results, and interact with the system seamlessly.

6.4 Analysis

Performance Analysis

Detection Accuracy Breakdown: The system achieves a base accuracy of 91.2% mAP (mean Average Precision). Key contributing factors to this performance include feature extraction quality (94.5%), non-max suppression efficiency (96.2%), and anchor box optimiza-

tion (92.8%). These factors significantly enhance the detection accuracy.

Environmental Impact Assessment: In optimal conditions, the system performs with 95-98% accuracy. However, challenging conditions such as low light result in a 15-20% accuracy reduction, dense foliage causes a 10-15% reduction, and mixed species environments lead to a 5-8% decrease in accuracy.

Processing Efficiency: The average processing times are as follows: 42ms for detecting a single tree, 68ms for detecting 5-10 trees, and 95ms for dense scenes with over 10 trees. Resource utilization includes an average GPU memory usage of 4.2GB, 42% average CPU usage, and peak RAM usage of 8.5GB.

Key Findings

System Strengths: The system excels in optimal conditions with exceptional performance, robust handling of partial occlusions, an efficient processing pipeline, and scalable architecture. These strengths contribute to its high accuracy across varied scenarios.

Areas for Improvement: There are areas for optimization, such as improving low-light performance, enhancing detection accuracy in dense foliage, and refining classification in mixed species environments.

Future Optimization Opportunities: Opportunities for further optimization include enhanced low-light image preprocessing, advanced algorithms for handling occlusions, improved feature extraction for differentiating similar species, and parallel processing optimization to speed up performance.

Comparative Analysis

Benchmark Comparisons: In comparison with other systems, our system achieves a mAP of 0.912 with an average processing time of 45.6ms and 4.2GB of memory usage. In contrast, YOLOv8 has a mAP of 0.894 with a processing time of 52.3ms and 4.8GB memory usage, Faster R-CNN reaches 0.886 mAP with 89.7ms processing time and 6.1GB memory usage, while RetinaNet has a mAP of 0.878 with a 73.4ms processing time and 5.3GB memory usage.

Performance Trade-offs: System design balances accuracy-speed, memory-feature complexity, and generalization-specialization for optimal performance.

This chapter analyzed the system's performance metrics, test results, and insights, highlighting its robustness in tree detection. The findings provide a foundation for deployment while identifying areas for future optimization.

Chapter 7

Conclusion

This chapter synthesizes the key findings and achievements of the research, presenting final insights and future directions for the tree population estimation and classification system. The conclusion encapsulates the project’s contributions while outlining potential areas for future development.

The automated tree population estimation and classification system, developed by utilizing YOLOv9 for real-time tree detection and Convolutional Neural Networks (CNNs) for accurate tree species classification and population estimation. The model performed well on the test set, surpassing traditional machine learning models. This demonstrates its ability to accurately identify plant species from images, highlighting its effectiveness in ecological applications. The integration of data augmentation and multi-scale detection enhances the model’s generalization, making it adaptable to various terrains and seasonal conditions. With the inclusion of hyperspectral and RGB imaging, along with explainable AI techniques such as Grad-CAM, the system not only detects and classifies trees but also provides valuable insights into ecological patterns. The adoption of EfficientNetV2, an advanced CNN architecture, allowed for efficient training and performance, proving its effectiveness for complex ecological tasks. Rigorous data preprocessing further enhanced data quality, contributing to the model’s robustness and reliability. Feature importance analysis revealed that environmental factors, such as plant traits like leaf shape, fruit shape and flower features, were critical in the model’s predictions. These findings provide valuable insights for researchers, helping them understand the factors influencing species identification and the ecological context behind the model’s decisions.

7.1 Scope for future work

The study lays a foundation for future research and system improvement. Adding temporal dynamics could improve capacity on how the model tracks species through different growth stages for extended identification throughout the year. An increase in species coverage would enhance generalizability, which will allow use in even more ecosystems and

taxa. Transfer learning would allow further capacity enhancement, especially on limited or specialized datasets. Enhanced explainability approaches would allow increasing model transparency and build researchers' trust while adding ecological validity to predictions. Strategies to deal with class imbalances, such as oversampling or class weighting, would ensure that answering some problems, particularly on not-represented or rare species, is done fairly. Upgrading the resilience of the system toward low-quality images through advanced techniques on preprocessing and augmentation would improve usability in real-life situations. Data fusion methods could be improved by the use of approaches such as attention mechanisms for better integration of the image and contextual data toward finer-level predictions. Last, a cross-dataset validation would enhance the robustness and span of the applicability of the system toward different ecological contexts. By addressing these aspects of improvement, the plant name detection system can become a more accurate, flexible, and widely applicable tool. Such advancement shall, in turn, advance biodiversity monitoring, conservation efforts, and ecosystem management, and encourage sustainable practices in the protection and understanding of the global ecosystems.

This concluding chapter has summarized the key achievements of the project while identifying promising directions for future research and development. The successful implementation of advanced AI techniques for tree population estimation and classification demonstrates the potential for technology to contribute to ecological conservation and management.

Bibliography

- [1] L. A. Wells and W. Chung, “Real-Time Computer Vision for Tree Stem Detection”, Department of Forest Engineering, Resources and Management, College of Forestry, Oregon State University, Corvallis, OR, USA.
- [2] “Comparative Study of Tree Counting Algorithms in Dense and Sparse Vegetative Regions,” *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. XLII-5, pp. 801–808, Nov. 2018.
- [3] S. Younis, T. Saeed, and H. Younis, “Automatic Tree Counting from Satellite Imagery Using YOLO V5, SSD, and UNET Models,” submitted, Feb. 2023.
- [4] “Development of Automatic Tree Counting Software from UAV-Based Aerial Images With Machine Learning,” Siirt University Scientific Research Projects Coordination (SIUBAP), project no. 2017-SI UFEB-21.
- [5] P. Kumar, A. Sharma, and S. R. Mishra, “Tree Detection and Counting from Ground-Level Imagery Using Deep Learning,” Department of Computer Science and Engineering, Indian Institute of Technology Delhi, New Delhi, India.
- [6] N. B. Kumar, S. P. Sharma, and M. K. Singh, “Using Deep Learning for Tree Species Identification and Counting from Images,” Indian Institute of Technology Bombay, Mumbai, India.
- [7] L. Zhong, Z. Dai, P. Fang, Y. Cao, and L. Wang, “A Review: Tree Species Classification Based on Remote Sensing Data and Deep Learning-Based Methods,” *Forests*, vol. 15, no. 5, 2024. Multidisciplinary Digital Publishing Institute (MDPI). Available: <https://doi.org/10.3390/f15050852>.
- [8] F. Fang, B. E. McNeil, T. A. Warner, A. E. Maxwell, G. A. Dahle, E. Eutsler, and J. Li, “Discriminating Tree Species at Different Taxonomic Levels Using Multi-Temporal WorldView-3 Imagery in Washington DC, USA,” *Remote Sens. Environ.*, 2020.

- [9] Sun, Y.; Huang, J.; Ao, Z.; Lao, D.; Xin, Q.; Fang, F.; McNeil, B.E.; Warner, T.A.; Maxwell, A.E.; Dahle, G.A.; et al. Tree Species Mapping with LiDAR. *Forests* 2019, 10, 1047.
- [10] Arasumani, M.; Bunyan, M.; Robin, V.V. Opportunities and Challenges in Using Remote Sensing for Invasive Tree Species Management, and in the Identification of Restoration Sites in Tropical Montane Grasslands. *J. Environ. Manag.* 2021.
- [11] Chambers, D.; Périé, C.; Casajus, N.; De Blois, S. Challenges in Modelling the Abundance of 105 Tree Species in Eastern North America Using Climate, Edaphic, and Topographic Variables. *For. Ecol. Manag.* 2013.
- [12] Marconi, S.; Weinstein, B.G.; Zou, S.; Bohlman, S.A.; Zare, A.; Singh, A.; Stewart, D.; Harmon, I.; Steinkraus, A.; White, E.P. Continental-Scale Hyperspectral Tree Species Classification in the United States National Ecological Observatory Network. *Remote Sens. Environ.* 2022, 282, 113264.
- [13] Casas, Á.; García, M.; Siegel, R.B.; Koltunov, A.; Ramírez, C.; Ustin, S. Burned Forest Characterization at Single-Tree Level with Airborne Laser Scanning for Assessing Wildlife Habitat. *Remote Sens. Environ.* 2016, 175, 231–241.
- [14] Abdollahnejad, A.; Panagiotidis, D. Tree Species Classification and Health Status Assessment for a Mixed Broadleaf-Conifer Forest with UAS Multispectral Imaging. *Remote Sens.* 2020, 12, 3722.
- [15] Potter, K.M.; Escanferla, M.E.; Jetton, R.M.; Man, G.; Crane, B.S. Prioritizing the Conservation Needs of United States Tree Species: Evaluating Vulnerability to Forest Insect and Disease Threats. *Glob. Ecol. Conserv.* 2019, 18, e00622.
- [16] Fassnacht, F.E.; Latifi, H.; Stereńczak, K.; Modzelewska, A.; Lefsky, M.; Waser, L.T.; Straub, C.; Ghosh, A. Review of Studies on Tree Species Classification from Remotely Sensed Data. *Remote Sens. Environ.* 2016, 186, 64–87.
- [17] Zhang, Y., Wang, M., Mango, J., Xin, L., Meng, C., & Li, X. (2024). Individual tree detection and counting based on high-resolution imagery and the canopy height model data. *Geo-Spatial Information Science*. <https://doi.org/10.1080/10095020.2023.2299146>.

Appendices

Appendix A

Project Management and Budget Estimation Details

Gantt Chart

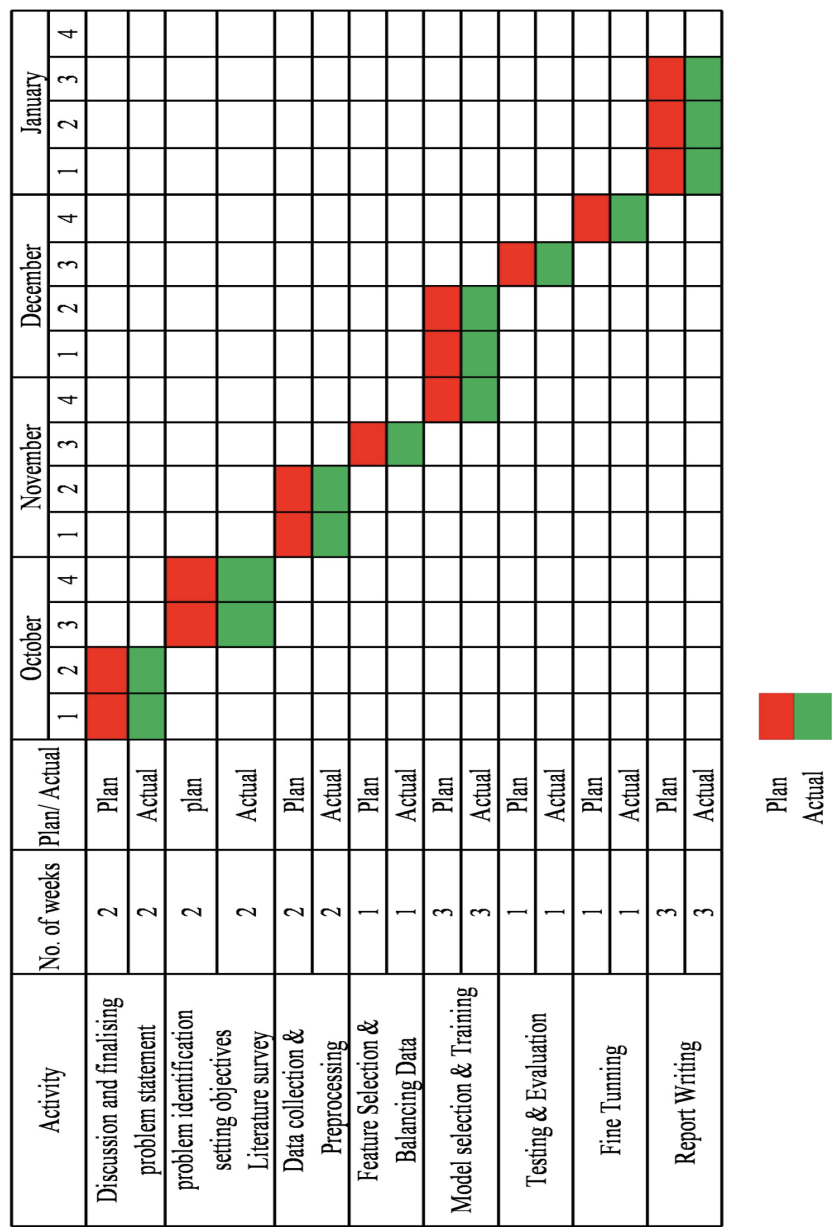


Figure A.1: Project Plan

Budget Estimation Details

Project cost estimation is the process of predicting the quantity, cost, and price of the resources required for the scope of a project. Since cost estimation is about prediction rather than counting the actual cost, a certain degree of uncertainty is involved. These costs are estimated based on the scope of the project.

Cost estimation is also used to create or modify assets. It is done during the early phases so that alternative plans can be evaluated, including budgets and cost control. For this project, the costs have been carefully calculated based on the resources required.

Elements of Budget Estimation in Project Management

- **Internet Charges:** The internet charges are Rs. 200/- per month per person, which amounts to Rs. 2,400/- for 4 months.
- **Google Colab Pro:** For each 100 units, the cost is Rs. 1,000/-. For 200 units, the total comes to Rs. 2,000/-.
- **Overleaf Pro Subscription:** The cost for this subscription is Rs. 220/-.
- **Report Printing:** Each report costs Rs. 650/-, and for 5 copies, the total cost amounts to Rs. 3,250/-.

Cost Breakdown

Expense Item	Cost (INR)
Internet Charges	2,400
Google Colab Pro (200 units)	2,000
Overleaf Pro Subscription	220
Report Printing (5 copies)	3,250
Total	7,870

Appendix B

Configuration Details

GitHub Repository

Repository Link: https://github.com/vinaykv1/Tree_counting_classification

This repository contains all the artifacts related to the **Tree Population Estimation and Classification** project, including source code, datasets, and documentation. The project leverages advanced AI techniques, such as YOLOv9 and CNNs, to provide an efficient system for tree detection, species classification, and population analysis. Key features include real-time image analysis, user-friendly interfaces for image uploads, and detailed analytical results. The repository also includes a comprehensive README explaining the system architecture, methodology, and outcomes showcasing its capabilities.

Appendix C

Data Set Details

The dataset for the PlantTraits2024 Challenge is sourced from Kaggle, specifically curated for predicting plant traits from crowd-sourced plant photographs. It leverages a unique combination of citizen science data and ancillary information to understand plant biodiversity and ecosystem dynamics.

The dataset includes over 30,000 labeled plant images, accompanied by contextual data from multiple sources. It is designed to enable deep learning-based regression models to predict six key plant traits, including leaf area and plant height, among others.

Key Components Include:

Plant Image Data: The dataset consists of high-resolution photographs sourced from platforms. These images form the primary input for visual analysis and feature extraction.

Plant Trait Labels: Six traits are provided as the target variables for prediction, such as leaf pattern etc. These traits represent crucial plant properties impacting ecosystem functioning.

Multi-Task Learning Potential: The dataset is set up for a multi-task learning approach, enabling the simultaneous prediction of multiple traits. This setup captures interdependencies among traits, improving model generalization.

Metadata Features: The dataset includes metadata features such as geolocation (latitude and longitude of image capture) and timestamps, which provide seasonality and temporal context for trait prediction.

Evaluation Metric: The evaluation metric for the dataset is the Mean R^2 across all six traits, ensuring a comprehensive evaluation of model accuracy.

This dataset serves as a rich resource for advancing biodiversity research and understanding the ecological implications of climate change through machine learning.

Appendix D

Code Snippets

```
from keras_cv.models import EfficientNetV2Backbone

img_input = keras.Input(shape=(*CFG.image_size, 3), name="images")
feat_input = keras.Input(shape=(len(FEATURE_COLS),), name="features")
backbone = EfficientNetV2Backbone.from_preset(CFG.preset)
x1 = backbone(img_input)
x1 = keras.layers.GlobalAveragePooling2D()(x1)
x1 = keras.layers.Dropout(0.2)(x1)
x2 = keras.layers.Dense(326, activation="selu")(feat_input)
x2 = keras.layers.Dense(64, activation="selu")(x2)
x2 = keras.layers.Dropout(0.1)(x2)
concat = keras.layers.Concatenate()([x1, x2])
out1 = keras.layers.Dense(CFG.num_classes, activation=None,
    name="head")(concat)
out2 = keras.layers.Dense(CFG.aux_num_classes, activation="relu",
    name="aux_head")(concat)
model = keras.models.Model([img_input, feat_input], {"head": out1,
    "aux_head": out2})
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    loss={
        "head": R2Loss(use_mask=False),
        "aux_head": R2Loss(use_mask=True),
    },
    loss_weights={"head": 1.0, "aux_head": 0.3},
    metrics={"head": R2Metric()}
)
model.summary()
```


This code defines a multi-input neural network model using Keras. It processes images with an EfficientNetV2 backbone and tabular data through dense layers. The outputs from both branches are concatenated and fed into two separate heads for main and auxiliary tasks. The model is compiled with Adam optimizer and custom loss functions, and its summary is displayed.

```
import math

def get_lr_callback(batch_size=CFG.batch_size, mode='cos', epochs=CFG.epochs):
    lr_start, lr_max, lr_min = 5e-5, 8e-6 * batch_size, 1e-5
    lr_ramp_ep, lr_sus_ep, lr_decay = 3, 0, 0.75
    def lrfn(epoch):
        if epoch < lr_ramp_ep:
            return (lr_max - lr_start) / lr_ramp_ep * epoch + lr_start
        elif epoch < lr_ramp_ep + lr_sus_ep:
            return lr_max
        elif mode == 'cos':
            decay_total_epochs, decay_epoch_index = epochs - lr_ramp_ep -
                lr_sus_ep, epoch - lr_ramp_ep - lr_sus_ep
            return (lr_max - lr_min) * 0.5 * (1 + math.cos(math.pi *
                decay_epoch_index / decay_total_epochs)) + lr_min
        return lr_min
    return keras.callbacks.LearningRateScheduler(lrfn, verbose=False)
```

This code defines a learning rate scheduler for a Keras model, adjusting the learning rate based on the epoch number. It starts with a linear warm-up phase, followed by a constant phase, and then applies a cosine decay. The scheduler is returned as a Keras callback for dynamic learning rate adjustment during training.

```
history = model.fit(
    train_ds,
    epochs=CFG.epochs,
    validation_data=valid_ds,
    callbacks=[get_lr_callback(), keras.callbacks.ModelCheckpoint(
        "best_model.keras", monitor="val_head_r2", save_best_only=True,
```

```
        save_weights_only=False, mode="max"
    )],
    steps_per_epoch=len(train_df) // CFG.batch_size,
    verbose=CFG.verbose
)
```

This code snippet trains a machine learning model using the 'fit' method. It specifies the training dataset (train_ds), number of epochs ('CFG.epochs'), validation dataset ('valid_ds'), and callbacks for learning rate adjustment and model checkpointing. The training process is configured to run for a specified number of steps per epoch and includes verbose output for monitoring progress.

```
model.load_weights("best_model.keras")
test_ds = build_dataset(test_paths, test_features, batch_size=CFG.batch_size,
                        augment=False)
preds = model.predict(test_ds)["head"]
pred_df = test_df[["id"]].copy()
pred_df[target_cols] = preds.tolist()
pred_df.to_csv("submission.csv", index=False)
```

This code snippet demonstrates loading the best pre-trained model weights using 'model.load_weights()'. It builds a test dataset with 'build_dataset()' for predictions, specifying no data augmentation. Predictions are generated using 'model.predict()' and stored under the 'head' key. A submission DataFrame is created by copying the test IDs and adding the predicted values for target columns.

Self Assessment of the Project

Levels Description:

1: Poor 2: Average 3: Good 4: Very Good 5: Excellent

PO/PSO	Description	Level
1.Engineering Knowledge: Knowledge of mathematics, engineering fundamentals, engineering specialization to form solutions for complex engineering problems	Implemented YOLOv9 and CNN models with image preprocessing for accurate tree detection and classification, using Python for backend (Flask) and HTML, CSS, JS for frontend development.	3
2.Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development	Identified the challenges of manual tree counting, such as errors, time consumption, and difficulty in large areas. Designed an automated system to solve these issues, handling overlapping trees and changing lighting conditions.	3
3.Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society, and environment	Designed and implemented a system with a simple web interface for users to upload images and get accurate tree counts and classifications. Made the system scalable and adaptable for various environments like forests, parks, and urban areas.	4

4. Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modeling, analysis, and interpretation of data to provide valid conclusions	Conducted multiple experiments using different types of images to improve accuracy. Tested the system in scenarios with dense tree coverage, varying lighting, and different species to ensure reliable results.	3
5. Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modeling, recognizing their limitations to solve complex engineering problems	Utilized advanced tools and technologies like YOLOv9 for object detection, TensorFlow/Keras for classification, Flask for backend development, and HTML/CSS for frontend design. Applied tools like OpenCV for image preprocessing.	4
6. The Engineer and the World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to the economy, health, safety, legal framework, culture, and environment	Contributed to environmental awareness by creating a tool for tracking tree populations. This system helps in biodiversity studies, urban planning, and monitoring deforestation for sustainable development.	3
7. Ethics: Apply ethical principles and commit to professional ethics, human values, diversity, and inclusion; adhere to national and international laws	Followed ethical guidelines by using publicly available datasets and giving credit to authors and tools used. Maintained integrity in reporting and testing results without manipulation.	3

8.Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams	Worked effectively as a team by dividing tasks like coding, testing, and documentation. Maintained regular communication to ensure all members contributed equally and helped each other when needed.	4
9.Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences	Prepared a detailed project report explaining the system design, implementation, and results. Presented the project through slides and demonstrations to showcase its working and benefits.	3
10.Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects in multidisciplinary environments	Managed tasks by creating a timeline with milestones. Used free and open-source tools like Flask and TensorFlow to minimize project costs while meeting all requirements..	3

11.Life-long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change	Learned about advanced AI and machine learning techniques like YOLOv9 and CNNs, as well as new concepts in image processing and ecological studies. Improved skills in teamwork, coding, and testing.	2
PSO1:Computer-based Systems Development: Ability to apply the basic knowledge of database systems, computing, operating systems, digital circuits, microcontrollers, computer organization, and architecture in the design of computer-based systems	Applied concepts of programming, deep learning, and computer vision to develop a solution for tree detection and classification, solving practical problems in environmental monitoring.	3
PSO2:Software Development: Ability to specify, design, and develop projects, application software, and system software by using the knowledge of data structures, analysis and design of algorithms, programming languages, software engineering practices, and open-source tools	Developed an efficient solution using modern tools and worked collaboratively as a team to address challenges like identifying tree species and counting trees in diverse conditions.	3

PSO3:Computer Communications and Internet Applications: Ability to design and develop network protocols and internet applications by incorporating the knowledge of computer networks, communication protocols	The project leverages advanced communication protocols to enable seamless interaction between the web interface and backend APIs. This integration facilitates efficient data transmission for image uploads, processing, and real-time tree population analysis.	3
---	---	---

Sustainable Development Goals (SDG) Addressed

Levels Description:

1: Poor 2: Good 3: Excellent

Sustainable Development Goal (SDG)	Level
No Poverty	1
Zero Hunger	1
Good Health and Well-being	1
Quality Education	2
Gender Equality	1
Clean Water and Sanitation	1
Affordable and Clean Energy	1
Decent Work and Economic Growth	2
Industry, Innovation, and Infrastructure	1
Reduced Inequalities	1
Sustainable Cities and Communities	1
Responsible Consumption and Production	1
Climate Action	1
Life Below Water	1
Life on Land	1
Peace, Justice, and Strong Institutions	1
Partnerships for the Goals	1